文章编号:1001-506X(2023)07-1938-19 网址:www. sys-ele. com

超维计算概念、应用及研究进展

刘文波[1,2],姚翼荣[1,2],张 弓[3,4,*],胡 文[3,4]

(1. 南京航空航天大学自动化学院,江苏 南京 211106;2. 高速载运设施的无损检测监控技术工业和信息化部重点实验室,江苏 南京 211106;3. 南京航空航天大学电子信息工程学院,江苏 南京 211106;4. 雷达成像与微波光子技术教育部重点实验室,江苏 南京 211106)

摘 要:超维计算是一种受大脑工作机制启发的新兴认知模型,使用信息的高维、随机、全息分布式表示作为处理对象,具有低运算成本、快速学习过程、高硬件友好性、强鲁棒性、不依赖大数据和优异的模型可解释性等优势,在分类识别、信号处理、多任务学习、信息融合、智能决策等领域有着良好的应用前景。近年来,超维计算受到的关注量持续增加,展现出巨大的发展潜力,为研究人员提供了一种新选择。本文详细介绍了超维计算的发展历史、基本原理和模型框架,给出超维计算的典型应用实例,并对超维计算现阶段存在的问题和未来可能的发展方向进行了探讨。

关键词:超维计算;分类识别;分布式表示;机器学习;研究进展

中图分类号:TP 181 **文献标志码**:A **DOI**:10.12305/j.issn.1001-506X.2023.07.04

Concept, application, and research progress of hyperdimensional computing

LIU Wenbo[1,2], YAO Yirong[1,2], ZHANG Gong[3,4,*], HU Wen[3,4]

(*1. College of Automation Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing 211106, China; 2. Key Laboratory of Non-Destructive Testing and Monitoring Technology for High-Speed Transport Facilities, Ministry of Industry and Information Technology, Nanjing 211106, China; 3. College of Electronic and Information Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing 211106, China; 4. Key Laboratory of Radar Imaging and Microwave Photonics, Ministry of Education, Nanjing 211106, China*)

Abstract: Hyperdimensional computing (HDC) is an emerging brain-inspired cognitive model. It uses high-dimensional random holographic distributed representations of different entities as processing objects and has many advantages such as low computational cost, superfast learning process, high hardware-friendliness, strong robustness, independent of data volume, and excellent model interpretability. HDC is a promising method in many fields, such as target recognition, signal processing, multi-task learning, information fusion, and smart decision-making. In recent years, HDC has received increasing attention and shown great development potential, which provides researchers with a new choice. This paper introduces the development history, basic principles, and the specific model architecture of HDC, presents some typical application examples, and discusses the existing deficiencies and possible future directions of HDC.

Keywords: hyperdimensional computing (HDC); category recognition; distributed representation; machine learning; research progress

收稿日期:2022-03-14; **修回日期**:2022-07-18; **网络优先出版日期**:2022-08-08。

网络优先出版地址:http://kns.cnki.net/kcms/detail/11.2422.TN.20220808.1638.004.html

基金项目:国家重点研发计划(2018YFB2003304);国家自然科学基金(62271255,62271286,61871218);上海航天科技创新基金(SAST2021-026);南京航空航天大学前瞻布局科研专项;雷达成像与微波光子技术教育部重点实验室开放课题基金(NJ20220008,NJ20210008)资助课题

*通讯作者.

引用格式:刘文波,姚翼荣,张弓,等. 超维计算概念、应用及研究进展[J]. 系统工程与电子技术,2023,45(7):1938-1956.

Reference format: LIU W B, YAO Y R, ZHANG G, et al. Concept, application, and research progress of hyperdimensional computing[J]. Systems Engineering and Electronics, 2023, 45(7): 1938-1956.

0 引　言

如何使计算机具有和人脑类似甚至更好的记忆、学习、联想、认知和思考能力，是长期以来广大研究人员共同希望解决的难题之一。神经科学在寻求解释人类神智活动的同时，也为机器学习领域提供了丰富的灵感来源，并由此衍生出了一系列类人脑人工智能算法。例如，从神经细胞间信息传递模型抽象出的单层感知机[1]和在其基础上发展的深度神经网络[2]，假定神经元突触相互作用是线性累加的相关矩阵存储器(correlation matrix memories，CMM)[3-4]，受视网膜神经元感受野启发的卷积神经网络[5]，借鉴大脑激活稀疏性的稀疏编码[6]，以及模拟大脑如何在嘈杂环境中将注意力集中到某个特定声音的独立成分分析(independent component analysis，ICA)算法[7]等。其中，神经网络在被提出初期，由于其较大的模型规模和当时有限的计算机算力，模型训练困难，准确率有限。直到 2012 年基于深度卷积神经网络的 AlexNet 模型[8]取得突破性进展后，神经网络的相关研究在图像识别[9]、自然语言处理[10]、语音识别[11]等诸多领域得到了广泛应用，以神经网络为代表的连接主义模型迅速成为了人工智能领域的研究热点。

随着需要处理的任务难度不断上升，神经网络模型不断向着更深、更广、更复杂的趋势发展。但是，这一趋势直接使神经网络的模型规模越来越庞大。目前，常用的神经网络模型，需要训练的参数量一般都在百万以上。巨大运算量带来的漫长训练时间对硬件设备的精度和性能要求较高，也需要更多的能耗和更好的模型稳定性。较低的硬件友好性也使神经网络很难在物联网、嵌入式系统等小型平台上完成实时片上训练。此外，神经网络模型的训练依赖大数据驱动、无指标调参和浮点数迭代运算，而大脑产生认知并执行决策时所需样本很少，快速联想能力强且容错性高，两者存在着本质差异。虽然在很多实际应用中，黑匣式神经网络模型做出的决策判断取得了良好的识别效果，但目前神经网络模型提取的特征和产生决策的具体逻辑仍然在语义层面难以理解，并且缺乏标准的数学工具定量评估模型的泛化能力和实际学习能力，其可信度和可解释性有限。大量工作围绕解决神经网络算法存在的上述问题开展，例如减少样本需求并加速训练过程的小样本学习[12]、简化模型结构的剪枝技术[13]、增加模型可解释性的可视化技术[14-15]等。

然而，这些在神经网络模型中需要通过额外处理才能在特定情况下实现的特性，直接且自然地存在于超维计算基本架构中[16]。因此，虽然超维计算的理论雏形早在 20 世纪 80 年代就被提出[17]，在近年来也作为一种新兴技术重新得到了全球范围内的关注和系统性的研究。

超维计算[18]同样是理论神经科学与计算机科学交叉领域的产物。与上文提到的神经网络等人工智能算法相比，相同之处在于其都将信息的高维分布式表示作为数据的表示方式或处理过程中的基本环节。生物学中的相关研究表明，许多生物的感觉系统中都有一个将相对低维的感觉输入信号转化为高维稀疏表示的器官，并使用此高维表示实现后续的认知行为[19-20]，此特点与这些算法中高维的概念相符。不同之处在于，超维计算将记忆的产生和回忆看作是一种高维的全局随机映射与相似度匹配，类似大脑中海马体将短时记忆信息经过加工转化后形成大脑皮层长时记忆的方式[21]，将新信息通过联想、筛选和整合，添加至已有的信息编码中，这与神经网络等连接主义模型中的多层复合数值运算不同。

在超维计算中，每一个实体都被编码映射为一个超维向量，即一个维度为数千、数万或更高的高维矢量。实体的信息被全息地分布在超维向量中的每个元素上，每个元素之间几乎是独立的。在超维向量所在的超维空间中，通过对向量进行加法、乘法和排列等简单运算即可产生丰富且具有特殊性质的数学计算行为[16]，进而实现后续的高能效、高鲁棒性认知操作[22-25]。在实际应用中，超维向量的运算通常由简单高效的逐位二进制整数运算实现。模型不依赖于大数据驱动，对每个样本通过一次编码映射即可完成训练，使超维计算所需的运算成本和对硬件性能的要求极低，具有超快速的训练时间和数据刷新率。其硬件友好性使小成本片上训练成为可能，并且在获得新样本时，已有模型可以实时更新与学习。同时，由于超维计算中的运算都是可逆的，模型也具有良好的可验证性和可解释性。

总的来说，超维计算以其更快的学习过程、更低的延迟率、更高的能效、更贴合神经科学的鲁棒性、更轻量的模型规模、更少的样本需求和更多的可解释特征，成为一种极具前途的片上实时学习和识别方法[26]，从基本架构上直接解决了神经网络等人工智能算法存在的问题，作为一种新兴轻量级分类器提供了辅助甚至取代现有传统机器学习方法的可能性[27-28]。目前，有关超维计算的系统理论、工程应用以及与其他算法配合使用的研究仍然处在快速发展阶段。

超维计算已经在语音识别[29]、语言识别[30-31]、手写数字识别[32]、合成孔径雷达(synthetic aperture radar，SAR)图像识别[33]、人体姿态识别[26]、生物信号处理[34-36]、脱氧核糖核酸(deoxyribonucleic acid，DNA)模式匹配[37]、多任务学习[38]、多传感器信息融合[39-41]和脑机接口[42]等诸多领域得到了很好的应用。特别是在计算资源严格受限的设备上，例如树莓派[43](raspberry pi，RPi)、特定应用集成电路[44-45](application specific integrated circuit，ASIC)、现场可编程门阵列[46-47](field programmable gate array，FPGA)、可变电阻式存储器[48-49](resistive random-access memory，ReRAM)和相变存储器[50](phase change memory，PCM)，超维计算的优势将更加突出。

然而，目前国内文献中暂时还没有超维计算相关的研究内容，对此算法进行相关研究的国内团队也较少。因此，本文将围绕超维计算的发展历史、基本原理、典型应用

以及存在的问题和未来发展方向几个方面，对超维计算理论进行系统性地阐述、分析和研究，基于其研究进展，为超维计算在更多相关领域中的应用提供一定的可行性和可能性。

1 超维计算的发展历史

1.1 超维计算的理论雏形

与其他人工智能算法的目的相同，超维计算同样尝试构造一个类人脑的高维认知模型，使计算机能够完成记忆、学习、联想和认知等操作。

人工智能这一名称诞生在 1956 年的达特茅斯会议上，经过曲折且漫长的发展，人工智能算法如今取得了令人瞩目的成果。早期的高维认知模型以线性模型为主。1957 年，Rosenblatt[1] 用简单的加减法运算发明了感知机，通过二元线性分类器实现最简单形式的前馈式人工神经网络。1972 年 Anderson[3] 和 Kohonen[4] 提出了线性相关矩阵存储器理论，其基本思想类似感知机中神经元之间的信息传递方式和联结结构，假设神经元突触的相互作用是线性叠加的，试图通过一个存储矩阵，即权重矩阵，构造一组神经元与另一组神经元之间的线性映射关系，将需要分析处理的各种信息等同为权重矩阵的特征向量。然而对于复杂的认知行为来说，线性模型的效果有限，因此后续研究开始向非线性模型发展。

通过压缩函数可以将线性相关矩阵存储器算法中使用的输出向量转化为点吸引子的形式，实现模型的非线性化。著名的 Hopfield 网络[51] 采用的就是这种方式，其核心功能是将残缺输入数据通过联想关联重建为完整数据，具有较好的识别性能和鲁棒性。后续发展为玻尔兹曼机[52] 和深度置信网络[53]（deep belief network，DBN），对人工智能领域产生了深远影响。然而，Hopfield 网络的存储容量是有限的。例如，具有 N 个神经元的二进制 Hopfield 网络，其最大可存储模式为 2^N 个。随着存储数据量的增长，权重矩阵的尺寸也会成倍上升，这将导致模型需要占用更大的存储空间和计算资源。为解决这一问题，在 Hinton 等人[54] 围绕联想存储器展开的工作基础上，Kanerva[17] 在 1988 年提出了稀疏分布式存储理论，通过引入随机权值的方式，使用固定尺寸的权重矩阵就可以构造无限容量的联想存储器系统，这也被认为是超维计算存储结构的直接理论雏形之一。

1.2 同一起源与不同演变

同时期中，神经网络等连接主义模型也在感知机之后逐渐发展。1969 年，Minsky 等人[55] 在对感知机的分析中，提出两大限制连接主义模型发展的缺陷：① 基本感知机无法处理异或回路；② 当时有限的计算机计算能力不足以完成大型多层神经网络的训练。1974 年，Werbos 首次提出了使用误差反向传播[56] 训练神经网络的方法，有效解决了感知机无法处理异或回路的问题。1986 年，Rumelhart 等人[57] 将反向传播算法推广至多层神经网络，但是由于计算机性能限制，仍然难以实现网络的训练和计算。卷积、池化等目前连接主义人工智能算法中的常用概念在当时也被 Fukushima[5] 引入，以实现更好的特征提取和减少运算量的效果，陆续诞生了如 LeCun 等人在 1989 年提出的使用反向传播进行训练的卷积神经网络[58] 等经典模型。

为解决当时多层神经网络难以训练的问题，Hinton[59] 在 1990 年指出，神经网络在处理层次化结构的信息时必须简化其模式表示方法，提出用简化表示来表征部分-整体结构。简化描述框架中要求设计一些向量，每个向量作为一部分一起组成一个整体，并被压缩成与原始向量相同维度的单个向量，这个简化后的向量可以作为表示更大整体的一部分。同时，缩减必须是可逆的，才能在部分-整体层次结构中双向移动，但该文献中未给出具体的实现方法。Smolensky[60] 提出了一种分布式表示方法，称为张量积表示。这种方法通过外积运算实现变量与其对应数值的绑定，其特性满足了简化描述中所期望达到的部分性质。例如，张量积表示通过简化融合，将整体和部分的局部特殊情况添加至已有的结构化数据表示中构成一个整体，允许从简单的表示递归构造复杂的表示等。张量运算在之后也成为连接主义模型的重要基础之一。

但是，张量积是以增加表示的尺寸为代价将低阶的信息带至高阶中，没有实现简化的本质目的。这一问题可以由 Plate 在 1995 年提出的全息简化表示[61]（holographic reduced representation，HRR）解决。在 HRR 中，通过循环卷积的思想实现简化描述：两个 n 维实向量在经过元素级的加法、乘法、循环移位运算后仍然得到一个 n 维实向量，这个实向量将包含由张量积得到的 $n\times n$ 矩阵中的所有元素。以 HRR 为基础，使用不同的数据类型又衍生出了 Kanerva 提出的飞溅码[62]、Gayler 提出的一般化双极飞溅码[63] 等编码方式，但核心原理基本类似。2003 年，经过补充完善，Plate 全面阐述了全息简化表示理论[64]，对于复向量等各类情况也同样适用。通过全息简化表示，可以实现对各类组合结构的编码和解码，其中的有关概念与超维计算已经非常相似，基本奠定了超维计算的核心思想。

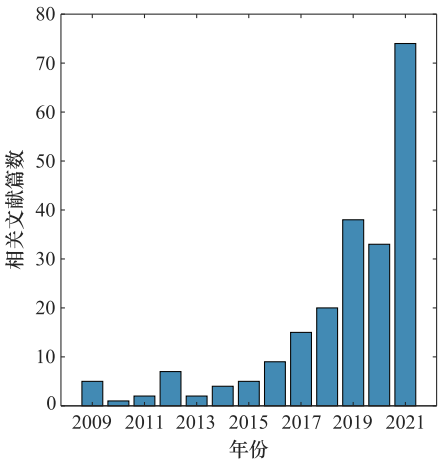
1.3 超维计算的系统性提出

2008 年，Levy[65] 将以高维、分布式数据表示作为识别基础的计算模型统称为矢量符号架构（vector symbolic architectures，VSA），涵盖了全息简化表示、飞溅码等诸多理论。矢量符号架构作为一种新型人工智能架构，提供了一种系统的方式来生成和操作各类信息的高维表示来完成认知操作。超维计算是矢量符号架构等早期模型的继承者，在硬件效率方面有着很强的附加优势[16]。超维计算中的大量概念和思想也同样适用于矢量符号架构模型，因此矢量符号架构也常被作为超维计算的别称之一。

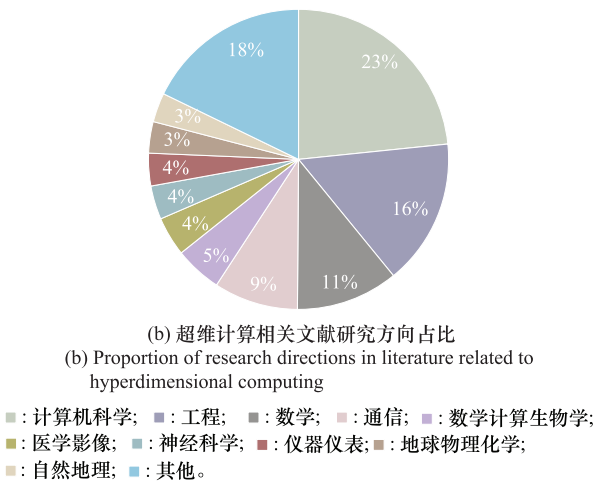
2009 年，Kanerva[18] 将矢量符号架构中的早期模型进行整合，以教程文章的形式首次正式提出超维计算的系统性理论与基本原理。Kanerva[18] 介绍了超维计算的模型结

构和模型背后的主要思想，对超维计算的性质、数据编码规则、具体运算方法和不同场景下的使用方式进行了详细说明，并给出了实际应用示例。在极低的运算成本和计算资源需求下，超维计算通过简单的算术运算和一次映射就可以在超维空间中实现对事物的高效识别、学习和认知，这一性质使超维计算在计算资源有限的应用场景中作为轻量级分类器展现出极大潜力。

截止至 2021 年底，科睿唯安 Web of Science 数据库中与超维计算相关的文献统计情况如图 1 所示。



(a) 超维计算相关文献各年份发表篇数
(a) Number of literature related to hyperdimensional computing



(b) 超维计算相关文献研究方向占比
(b) Proportion of research directions in literature related to hyperdimensional computing

图 1　超维计算相关文献统计情况

Fig. 1　Statistics of literature related to hyperdimensional computing

可以看出，自 2009 年 Kanerva 在文献[18]中正式提出超维计算的概念和原理以来，超维计算相关文献每年的发表篇数保持着快速上升趋势，并在近年来得到全球范围内研究人员越来越多的关注，未来一段时间内仍有着巨大的发展空间和研究价值。除计算机科学外，超维计算得到应用的研究方向还包括了工程、数学、通信、数学计算生物学、神经科学、医学影像、仪器仪表、地球物理化学等领域，适用情景非常丰富。

2　超维计算的背景知识

本节介绍超维计算中的一些背景知识，为后文中介绍超维计算的模型框架提供一些基本理论和方法。

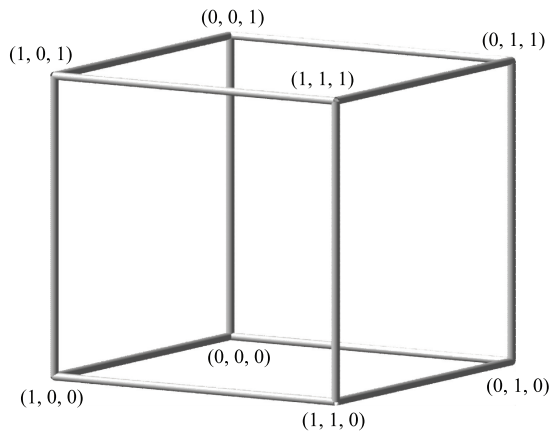
2.1　超维空间

超维计算利用超维空间中丰富且精妙的数学性质，将神经网络中的线性代数和概率思想与超维空间中的抽象代数和几何思想巧妙结合。

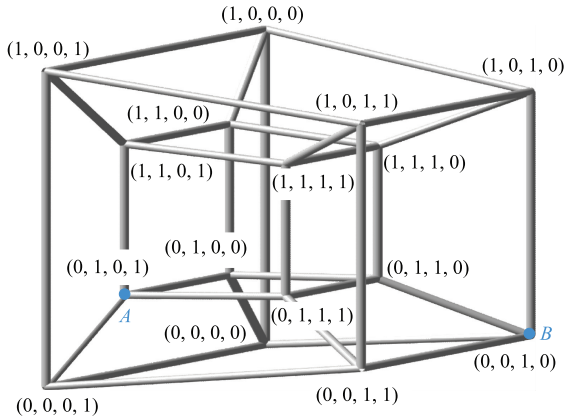
作为一种模拟大脑思维方式的认知模型，超维计算受到了大脑中全局映射记忆空间的启发。借鉴大脑中庞大规模的神经回路，超维计算通过对超维向量的操作来模拟大脑中的神经活动。超维向量所在的超维空间维数一般在数千、数万或更高，但是高于三维的空间对于三维世界中的人们来说是不可想象的，使用类比法有助于更好地理解超维空间的概念。

将一个点经过平移可以得到一条直线，这条直线将包含起点和终点两个点及其之间的直线；将这条直线继续平移则得到一个平面，该平面将包含原始直线、平移后的直线、两个端点各自平移得到的两条新直线、原始直线的两个点和平移后直线的两个点，共 4 条线(边)和 4 个点(角)。将平面继续平移可以得到一个立方体，该立方体将包含原始平面、平移后的平面、原始平面四条边各自平移得到的 4 个新平面、原始平面的 4 条线、平移后平面的 4 条线、原始平面 4 个点各自平移得到的 4 条新直线、原始平面的 4 个点和平移后平面的 4 个点，共 6 个面、12 条线(棱)、8 个点(顶点)。将立方体继续平移，可以得到一个四维超立方体，该四维超立方体所在的空间中将包含 8 个立方体、24 个面、32 条线(棱)和 16 个点(顶点)。更高维度的空间情况也可以继续以此类推得到。

超维空间的可视化只能通过不断的投影操作降低维度，直至得到其在二维平面上的投影才能实现，然而这个投影也很难反映超维空间的真实空间状态。以单位三维立方体和单位四维超立方体为例，设 d 为维度，图 2 为其在二维平面上的投影图。



(a) d=3时的单位三维立方体
(a) Three-dimensional unit cube when d=3



(b) d=4时的单位四维立方体
(b) Four-dimensional unit hypercube when d=4

图 2 d 维空间中的单位(超)立方体

Fig. 2 Unit (hyper-) cube in d-dimensional space

其中,图 2(a)很容易理解。将一个立方体放入三维笛卡尔坐标系中,则可以从(0,0,0)到(1,1,1)给其每个顶点分配一个唯一的坐标标号。但值得注意的是,这个标号仅用于区别开该三维空间中 2^3 个不同的点,每个点本身完全平等,即从任何一个顶点观察该三维立方体都是完全相同的。以此类推,如图 2(b)所示,可以用 4 维向量来区别开四维空间中的 2^4 个不同的点,这 16 个点在空间中也都完全平等。因此,设超维空间的维度为 d,一个 d 维超维向量实际上表示的是该 d 维空间中 2^d 个完全平等的空间点之一。在几何意义上,超维计算中的运算是对表示为超维向量的空间点的操作。

2.2 相似度衡量

超维计算使用超维向量作为基本运算对象,相似度是衡量超维向量之间关系的重要指标,也是超维计算模型编码和决策的重要判定依据。

虽然同一超维空间中的超维向量所表示的空间点是等价的,但不同点之间存在着不同的空间关系,直观表现在点与点之间的距离上。超维计算中对于相似度关系的衡量也通过向量之间的距离来定量表示,根据向量元素编码类型的不同,一般使用汉明距离和余弦距离计算。

汉明距离适用于采用二进制或双极等二元数据类型进行编码的超维向量相似度衡量中。采用此类编码方式的超维向量在每个向量元素上只有两种取值,因此在衡量两个超维向量 $\boldsymbol{A}$ 和 $\boldsymbol{B}$ 之间的关系时,逐位比较的结果也只有两种,即每一位的数值相同或不相同。汉明距离按此原理可以表示为

$$\mathrm{Ham}(\boldsymbol{A},\boldsymbol{B}) = \sum_{i=1}^{d} 1_{\boldsymbol{A}(i)\neq\boldsymbol{B}(i)} \tag{1}$$

式中:i 表示超维向量的第 i 位元素。可以看出,式(1)的算术意义是得到超维向量 $\boldsymbol{A}$ 和 $\boldsymbol{B}$ 之间不同位的个数。在几何意义上,汉明距离可以得到超维向量 $\boldsymbol{A}$ 和 $\boldsymbol{B}$ 之间的曼哈顿距离,即在超维空间中沿着超立方体的棱从 $\boldsymbol{A}$ 和 $\boldsymbol{B}$ 的最短距离。以图 2(b)中的 $\boldsymbol{A}$ 和 $\boldsymbol{B}$ 为例,其不同的位数和曼哈顿距离均为 3。为了避免式(1)计算出的数值大小受到超维空间维数 d 的影响,通常将式(1)归一化处理,使用归一化汉明距离衡量相似性:

$$H(\boldsymbol{A},\boldsymbol{B}) = \mathrm{Ham}(\boldsymbol{A},\boldsymbol{B})/d \tag{2}$$

如图 3 所示,如果在超维空间中任意选取一对采用二元数据类型进行编码的超维向量,其归一化汉明距离的概率密度函数将集中在 0.5 左右,且集中程度随着维数的增大而增大。根据统计,当 d=10 000 时,使用二进制编码或双极编码的一个超维向量与其他($2^{10\,000}-1$)个超维向量之间的归一化汉明距离中,仅有百万分之一小于0.476或大于0.524,仅有十亿分之一小于 0.470 或大于0.530[17]。这一特点表明,在概率上,几乎所有互相之间不存在关联的超维向量对之间的归一化汉明距离都为 0.5 左右,而更靠近 0 和更靠近 1 的归一化汉明距离只有通过人为设置或运算操作才可能出现。

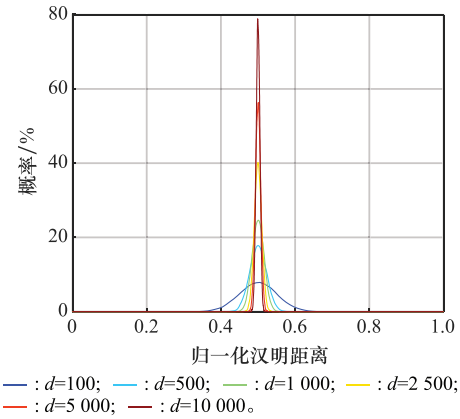


图 3 不同维度下超维向量之间归一化汉明距离的概率密度分布

Fig. 3 Probability density distribution of the normalized Hamming distance between hypervector pairs with different dimensionalities

因此在超维计算中,定义当两个超维向量之间的归一化汉明距离为 0.5 时,这两个超维向量相互(伪)正交;归一化汉明距离越靠近 0,说明两个超维向量之间的相似性越好;反之归一化汉明距离越靠近 1,说明正交性越好。仅当两个超维向量中的元素全部对应相同或不相同时,归一化汉明距离才会等于 0 或 1。

对于采用非二元数据类型进行编码的超维向量,其相似度一般使用余弦距离衡量。余弦距离计算两个向量之间的空间夹角大小:

$$\cos(\boldsymbol{A},\boldsymbol{B}) = \frac{\boldsymbol{A}\cdot\boldsymbol{B}}{|\boldsymbol{A}|\cdot|\boldsymbol{B}|} \tag{3}$$

与数学中向量的定义相同,超维向量之间的余弦距离越接近 0,说明它们之间的正交性越高;越接近 1,说明其相似性越高;当元素全部对应不相同时,余弦距离为−1。

当然,余弦距离也可以在汉明距离适用的情况中使用,但是可以明显看出,余弦距离的计算复杂度要高于汉明距离,其计算公式包括了高维度向量的内积、求模以及除法运算。相应地,这也需要相对较高的计算资源,运算效率也会略有下降。但相比于其他人工智能算法中的张量乘积、梯度求解等运算,余弦距离的运算量仍然是微不足道的。

2.3 基本运算

在超维空间中,通过对超维向量的运算操作可以产生丰富的数学行为。

超维计算中常用的基本运算有乘法、加法、排列 3 种，运算结果的维度与操作数保持一致。3 种基本运算的运算复杂度均为 $O(d)$，每种运算具有各自的性质，且乘法和排列运算是可逆的，加法运算是近似可逆的，具体介绍如下。

(1) 乘法

在超维计算中，乘法是一种由两个具有相同维数的超维向量得到一个乘积超维向量的运算操作。对于采用二进制编码的超维向量而言，乘法运算的计算方法与异或运算相同，运算符可以用符号⊕表示。例如，二进制编码的 8 维向量 $\boldsymbol{A}$ 和 $\boldsymbol{B}$ 在超维计算中乘法运算为

$$\begin{aligned}\boldsymbol{A}&=(1,0,0,1,1,0,1,1)\\ \boldsymbol{B}&=(0,0,1,1,0,1,1,0)\\ \hline \boldsymbol{A}\oplus\boldsymbol{B}&=(1,0,1,0,1,1,0,1)\end{aligned}\tag{4}$$

对于采用双极编码的超维向量而言，乘法运算与普通的数学乘法完全一致，将两个超维向量中的元素按位对应相乘即可，运算符可以用符号 * 表示。例如，双极编码的 8 维向量 $\boldsymbol{C}$ 和 $\boldsymbol{D}$ 在超维计算中乘法运算如下所示：

$$\begin{aligned}\boldsymbol{C}&=(1,-1,1,-1,-1,1,-1,1)\\ \boldsymbol{D}&=(1,-1,1,1,-1,-1,1,-1)\\ \hline \boldsymbol{C}*\boldsymbol{D}&=(1,1,1,-1,1,-1,-1,-1)\end{aligned}\tag{5}$$

由式(4)和式(5)可以发现，当二进制编码的超维向量与自身相乘时，得到的乘积超维向量将是一个元素值全为 0 的相同维度向量；当双极编码的超维向量与自身相乘时，得到的乘积超维向量将是一个元素值全为 1 的相同维度向量。依据该性质，可以很容易地得到超维计算中乘法运算的逆运算求解方式：

$$\boldsymbol{A}\oplus(\boldsymbol{A}\oplus\boldsymbol{B})=(\boldsymbol{A}\oplus\boldsymbol{A})\oplus\boldsymbol{B}=\text{zeros}(1,d)\oplus\boldsymbol{B}=\boldsymbol{B}\tag{6}$$

$$\boldsymbol{C}*(\boldsymbol{C}*\boldsymbol{D})=(\boldsymbol{C}*\boldsymbol{C})*\boldsymbol{D}=\text{ones}(1,d)*\boldsymbol{D}=\boldsymbol{D}\tag{7}$$

式中：zeros(1,d)和 ones(1,d)分别表示 d 维全 0 超维向量和 d 维全 1 超维向量。

如图 4 所示，以从 10 000 维空间中随机选择的两个二进制编码超维向量 $\boldsymbol{A}$ 和 $\boldsymbol{B}$ 为例，可以看到，乘积超维向量与其乘数、被乘数超维向量之间的归一化汉明距离集中在0.5左右，仍然保持正交。

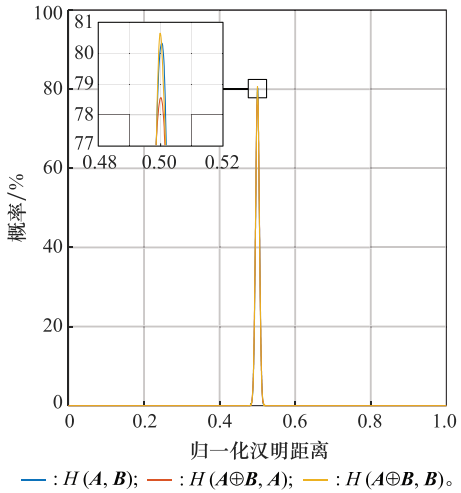


图 4　乘法运算中的相似度变化

Fig. 4　Similarity change in multiplication

在几何意义上，乘法运算相当于将超维空间中的两个点绑定后作为一个整体映射到空间中的另一个位置，过程中保持点与点之间的汉明距离[18]。这很类似于数学中自变量与因变量的绑定，或者是计算机中地址与数值的绑定。因此，乘法运算常被用来绑定一个地址-数值对。例如，将式(6)中的 $\boldsymbol{A}$ 看作地址，$\boldsymbol{B}$ 看作是需要和地址 $\boldsymbol{A}$ 绑定的数值，在经过乘法运算绑定后，可以通过类似取址运算的操作，将乘积超维向量与地址超维向量相乘实现指定地址中数值的恢复。

(2) 加法

加法在超维计算中是一种逐位的、近似可逆的运算，通过多个相同维数的超维向量相加得到一个和超维向量。设有 N 个加数超维向量 $\boldsymbol{X}_n(n=1,2,\cdots,N)$，相加得到和超维向量 $\tilde{\boldsymbol{R}}$ 的运算可以表示为

$$\tilde{\boldsymbol{R}}=\boldsymbol{X}_1+\boldsymbol{X}_2+\cdots+\boldsymbol{X}_N\tag{8}$$

若 $\boldsymbol{X}_n$ 采用二元数据类型进行编码，加法运算得到的和超维向量 $\tilde{\boldsymbol{R}}$ 很难继续保持二元形式，此时只能使用余弦距离衡量不同和超维向量之间的相似度。虽然非二元形式在一定程度上可以保留更多有效信息，实现更高的精度和准确度，但在某些严格追求低计算资源占用的情况下，标准化处理仍然是必要的。以采用二进制编码为例，将标准化运算记为$\lceil\cdot\rfloor$，运算方法如下：

$$\boldsymbol{R}(i)=\lceil\tilde{\boldsymbol{R}}(i)\rfloor=\begin{cases}0, & \tilde{\boldsymbol{R}}(i)/N<1/2\\ 1, & \tilde{\boldsymbol{R}}(i)/N>1/2\\ \text{随机}, & \tilde{\boldsymbol{R}}(i)/N=1/2\end{cases}\tag{9}$$

当奇数个超维向量相加时不会出现歧义，直接将和超维向量 $\tilde{\boldsymbol{R}}$ 中的每一位元素数值除以加数超维向量的总个数 N，通过对比该均值与 1/2 的大小关系进行阈值处理即可。但当偶数个超维向量相加时，在某些位上可能会出现加数超维向量中 0 的个数与 1 的个数完全相同的情况。此时，该位计算得到的均值将正好等于 1/2。实验表明，在超维计算中当维数较高时，这些位上数值是选择 0 还是选择1对整个加法运算几乎没有影响[27]，因此这些位在标准化处理中常采用随机选择的方式。给出 3 个二进制编码的 8 维向量进行加法运算的例子如下：

$$\begin{aligned}\boldsymbol{X}_1&=(0,0,1,1,0,1,0,1)\\ \boldsymbol{X}_2&=(1,0,1,0,1,0,0,1)\\ \boldsymbol{X}_3&=(1,1,0,1,1,0,1,1)\\ \hline \tilde{\boldsymbol{R}}&=(2,1,2,2,2,1,1,3)\\ \boldsymbol{R}&=(1,0,1,1,1,0,0,1)\end{aligned}\tag{10}$$

从式(10)中可以看出，超维计算加法运算在经过标准化处理后的功能与多人表决器相同，即选取每个位置上出现次数最多的数。双极编码的超维向量加法运算也基本相似，改用正负号作为阈值进行标准化处理即可。

仍然以 10 000 维空间为例，加法运算中超维向量之间的相似度变化情况如图 5 所示。其中，$\boldsymbol{X}_i$ 和 $\boldsymbol{X}_j$ 为采用二

进制编码两个超维向量，$\boldsymbol{R}=\lceil \boldsymbol{X}_1+\cdots+\boldsymbol{X}_i+\cdots+\boldsymbol{X}_j+\cdots+\boldsymbol{X}_N \rfloor$ 为经过标准化处理后的和超维向量，$\boldsymbol{Y}=\lceil \boldsymbol{Z}-\boldsymbol{X}_i \rfloor$ 为加法的逆运算，即减法运算，可以通过补码将减法转化为加法运算实现。但是，如果对标准化处理后的结果使用减法运算，由于信息在阈值处理中有所丢失，只能近似地恢复出原始数据，恢复出的数据与原始数据不完全相同但仍然存在一定的相似性。因此，超维计算中的加法运算也被称为是近似可逆的。

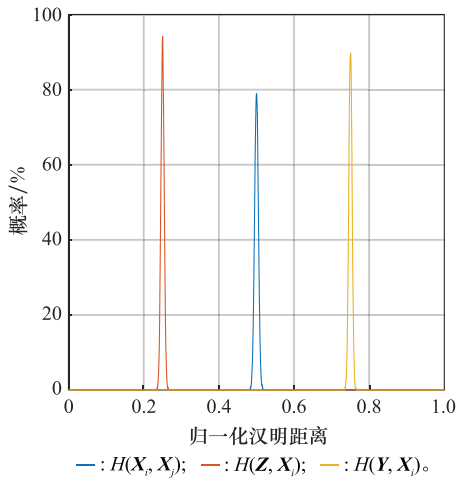


图 5 加法运算中的相似度变化

Fig. 5 Similarity change in addition

从图 5 中可以看出，与乘法运算前后不改变相似度的性质不同，和超维向量与其加数超维向量之间的归一化汉明距离向靠近 0 的方向移动，即保持较高的相似性。减法运算则相反，得到的差超维向量与其减数超维向量之间的正交性将更加显著。该性质表明，在加法运算中，和超维向量可以认为是由多个确定的加数超维向量作为元素构成的一个整体集合，集合中的每个元素独立且无序，加法运算结果保留了各个元素的信息并保持与每个元素相似。因此，加法运算常被用来表示一个具有多元素的集合。

(3) 排列

超维计算中的排列运算主要用于重新排列超维向量中各个元素的位置，一般只使用在采用二元数据类型进行编码的超维向量中。这种向量元素位置的变换可以表示为一个超维向量和一个变换矩阵的乘积。考虑到超维计算的硬件友好性和计算速度，排列运算一般使用循环移位实现。作为排列运算的一种特殊情况，移位运算在计算机或集成电路中极易实现且非常高效，因此也被作为排列运算最常见的实现方式。例如，记排列运算符为 Π，采用二进制编码的 8 维向量 $\boldsymbol{A}$ 的排列运算可以表示为

$$\begin{aligned}\boldsymbol{A}&=(1,0,0,1,1,0,1,1)\\ \overline{\Pi\boldsymbol{A}}&=(1,1,0,0,1,1,0,1)\end{aligned}\tag{11}$$

排列运算的性质与乘法基本相同，$\Pi\boldsymbol{A}$ 与 $\boldsymbol{A}$ 仍然保持正交。

由前文描述中可以发现，乘法运算和加法运算的结果都与操作数的顺序无关，满足交换律和结合率。以乘法为例，式(4)与式(5)显然满足交换律 $\boldsymbol{A}\oplus\boldsymbol{B}=\boldsymbol{B}\oplus\boldsymbol{A}$ 以及 $\boldsymbol{C}*\boldsymbol{D}=\boldsymbol{D}*\boldsymbol{C}$。但是，当引入额外的排列运算后，元素的先后顺序不同将产生不同的计算结果：$\Pi\boldsymbol{A}\oplus\boldsymbol{B}\neq\Pi\boldsymbol{B}\oplus\boldsymbol{A}$，$\Pi\boldsymbol{C}*\boldsymbol{D}\neq\Pi\boldsymbol{D}*\boldsymbol{C}$，且排列运算的逆运算 Π^{-1} 使用与 Π 相反的移位方向即可实现。这意味着排列运算具有保留元素先后顺序的独有功能。采用双极编码或在加法运算中，上述性质也同样成立。因此，排列运算常被用于保留序列、单词中的字母等关键顺序信息，进一步增强超维向量的信息表示能力。

3 超维计算的模型框架

本节给出具体的超维计算模型框架，阐述超维计算如何将样本编码为超维向量以及详细的训练和测试流程，并介绍一种超维计算中常用的模型完善和调整方法，即重训练方法。

基于第 2 节的背景知识，在超维空间中，通过对超维向量的操作可以产生丰富的计算行为，为编码、学习和认知更复杂的样本数据和信息提供基础。超维计算的模型不同于连接主义模型，在简单、快速、高效的同时，框架也相对固定，编码过程是影响最终的模型效果的核心环节。正如文献[33]中所说，超维计算的性能完全取决于模型架构的设计，而不是模型的具体训练过程，这也是超维计算不同于连接主义模型之处。

3.1 地址编码与数值编码

在超维计算中，任何形式的样本都需要编码为超维向量才能进行后续操作。样本在进行编码前的原始数据形式是多样的，例如信号、字符、图像等，样本通常不是由单一数值构成的，且样本中不同数值所在的地址信息也至关重要。在讨论具体的样本编码方式前，需要先介绍在超维计算中如何对地址和各地址中的数值进行编码。

地址是独立且唯一的，不同的地址在不同的样本中可能代表着不同的含义，如信号中的时间点、序列中的元素索引号、不同量纲信号的变量名称、图像中的像素位置等。因此，对于地址的超维向量编码也必须是唯一的，无重叠的，才可以与其代表的含义一一对应。利用在第 2.2 节中得出的结论：从超维空间中任意选择的超维向量将在概率上与其他超维向量保持正交。这意味着对于地址的编码，只需要从超维空间中随机选出一个与已有超维向量都不同的超维向量，向量中的各位元素服从独立同分布，那么这个地址超维向量就可以独立且唯一地作为一个地址的编码。

地址中存储的数值则与独立地址不同，数值可以代表个数、幅值、质量、灰度等信息，在同一范围内的数值之间并不是完全独立的，例如 0～5 V 的电压、50～100 g 的质量等。每个范围内通常包含着多个相互关联的数值，具体的离散点个数由采样精度决定。连续映射[35]是将同一范围内的多个数值映射到超维空间中的一种有效方法。

将范围在[MIN,MAX]之间的 L 个数值使用连续映射采用二元数据类型编码至 d 维空间的具体步骤如下。

步骤 1　从 d 维空间随机选择一个超维向量 $\boldsymbol{V}_1$ 代表区间中的最小值 MIN。

步骤 2　随机翻转前一个数值所对应超维向量中的 $d/2/(L-1)$ 位编码下一个数值，每一位只可翻转一次，不重复翻转。

步骤 3　重复第 2 步直至 L 个数值全部编码完成，最终得到 L 个数值超维向量，记为 $\boldsymbol{U}_l(l=1,2,\cdots,L)$。

从上述步骤中可以发现，$\boldsymbol{U}_1$ 与 $\boldsymbol{U}_L$ 之间不同的位数为 $d/2$，归一化汉明距离为 0.5，即 $\boldsymbol{U}_1$ 与 $\boldsymbol{U}_L$ 正交，而其他数值超维向量之间的正交性随着其所代表数值之差的增大而增大。图 6 展示的是基于连续映射将[0,255]内的 256 个灰度值采用二进制编码，编码为 256 个 10 000 维的数值超维向量，使用归一化汉明距离衡量任意两个数值超维向量之间的相似度关系。

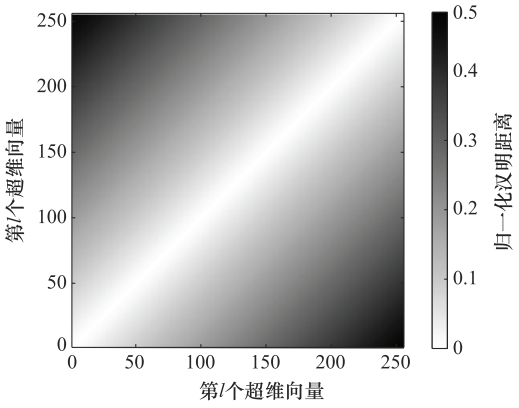


图 6　连续映射编码的数值超维向量之间的相似度

Fig. 6　Similarity between value hypervectors encoded by continuous mapping

在超维向量的编码、传输、存储等过程中，干扰信息不可避免地与有效信息一起进入超维向量中，但超维空间的超高维度对于噪声和杂波干扰有着较好的鲁棒性。假设有两个 d 维二进制编码的超维向量 $\boldsymbol{V}_a$ 和 $\boldsymbol{V}_b$ 是相似的，即其不同位数的占比小于 0.5，记为 $p<0.5$。如果由于各种原因导致 $\boldsymbol{V}_a$ 中的某些位发生了错误，将这些发生错误的位的占比记为 q，发生错误的 $\boldsymbol{V}_a$ 记为 $\boldsymbol{V}_{\text{noise}}$，那么 $\boldsymbol{V}_{\text{noise}}$ 将与 $\boldsymbol{V}_a$ 之间有 $q\cdot d$ 位不同元素，而 $\boldsymbol{V}_{\text{noise}}$ 与 $\boldsymbol{V}_b$ 之间的不同元素有 $(1-p)d\cdot q+(p\cdot d-p\cdot d\cdot q)=q\cdot d+(1-2q)p\cdot d$ 位，二者相差 $(1-2q)p\cdot d$ 位元素。因此，只要 $q<0.5$，则 $\boldsymbol{V}_{\text{noise}}$ 在超维计算中仍然会被认为是 $\boldsymbol{V}_a$，而不会被误判为与 $\boldsymbol{V}_a$ 相似的 $\boldsymbol{V}_b$，且当 $\boldsymbol{V}_a$ 和 $\boldsymbol{V}_b$ 之间不同位的占比 p 越大或者维数 d 更高时，区分度将会更加明显。

3.2　样本编码

在超维计算模型中，编码过程类似特征提取环节，编码得到的超维向量将唯一指代该样本，表征该样本的全部特征，并作为超维计算的输入。

现有的样本编码方法中，主要将数据按两种形式进行编码：记录型[35]和多元型[66]。这两种类型之间没有严格的划分并且可以相互转化，某些数据既可以作为记录型也可以作为序列型。一般来说，记录型样本编码侧重更高的编码精度，保留更多信息；而序列型样本编码侧重更快的运算速度，占用更少的计算资源。具体编码方式如下。

（1）记录型样本编码

在记录型样本编码中，数据被看作是由大量地址-数值对构成的集合。例如，语音信号可以看作是由一系列时间点作为地址，各个时间点的声音振幅作为数值构成的集合；图像可以看作是由一系列像素位置作为地址，各个像素位置中的像素值作为数值构成的集合。

根据前文的讨论，假设采用二进制编码方式，如果数据中需要进行编码的地址总个数为 M，通过从超维空间中随机选取 M 个超维向量对地址进行编码，记为 $\{\boldsymbol{H}_1,\boldsymbol{H}_2,\cdots,\boldsymbol{H}_M\}$。设各个地址中的取值范围为[MIN,MAX]中的 L 个离散数值，使用连续映射可以将这 L 个取值编码为数值超维向量 $\{\boldsymbol{U}_1,\boldsymbol{U}_2,\cdots,\boldsymbol{U}_L\}$。将每个地址中的数值对应的数值超维向量记为 $\{\boldsymbol{V}_1,\boldsymbol{V}_2,\cdots,\boldsymbol{V}_M\}$，其中，$\boldsymbol{V}_m\in\{\boldsymbol{U}_1,\boldsymbol{U}_2,\cdots,\boldsymbol{U}_L\}$ $(m=1,2,\cdots,M)$。

如图 7 所示，在记录型样本编码中，首先通过乘法运算将每个地址与其数值进行绑定构成地址-数值对，再通过加法运算将所有地址-数值对组合为一个集合，得到最终的样本超维向量。

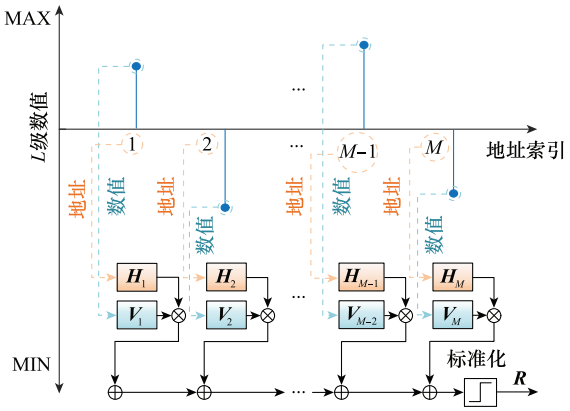


图 7　记录型样本编码

Fig. 7　Record-based encoding

该过程用公式可以表示为

$$\boldsymbol{R}=\lceil \boldsymbol{H}_1\oplus\boldsymbol{V}_1+\cdots+\boldsymbol{H}_M\oplus\boldsymbol{V}_M \rfloor \tag{12}$$

在不同的应用场景中，地址与数值分别代表的实际意义也是多样的。记录型样本编码与加法运算一样是近似可逆的。例如，若要从 $\boldsymbol{R}$ 中还原出地址 $\boldsymbol{H}_1$ 中的数值，只需要将地址超维向量 $\boldsymbol{H}_1$ 与 $\boldsymbol{R}$ 相乘：

$$\begin{aligned}\boldsymbol{H}_1\oplus\boldsymbol{R}&=\boldsymbol{H}_1\oplus\lceil \boldsymbol{H}_1\oplus\boldsymbol{V}_1+\cdots+\boldsymbol{H}_M\oplus\boldsymbol{V}_M \rfloor=\\ &\lceil \boldsymbol{H}_1\oplus\boldsymbol{H}_1\oplus\boldsymbol{V}_1+\cdots+\boldsymbol{H}_1\oplus\boldsymbol{H}_M\oplus\boldsymbol{V}_M \rfloor=\lceil \boldsymbol{V}_1+\text{noise} \rfloor\approx\boldsymbol{V}_1\end{aligned} \tag{13}$$

式中：除了第一项 $\boldsymbol{H}_1\oplus\boldsymbol{H}_1\oplus\boldsymbol{V}_1$ 以外的项，根据超维向量的正交性可以视为噪声，在高维度中对 $\boldsymbol{V}_1$ 的影响可以近似忽略[27]。

(2) 多元型样本编码

多元型样本编码将数据看作是由大量元素绑定形成，在编码过程中，每个元素的数值仍然通过连续映射进行编码，但各个元素的位置信息由排列运算保留和体现，这与记录型样本编码中使用地址超维向量的方式不同。

与上文保持一致，以采用二进制编码的情况为例，设数据中需要进行编码的地址总个数为 M，连续映射得到 L 个数值超维向量$\{\boldsymbol{U}_1,\boldsymbol{U}_2,\cdots,\boldsymbol{U}_L\}$。各地址中对应数值的超维向量记为$\{\boldsymbol{V}_1,\boldsymbol{V}_2,\cdots,\boldsymbol{V}_M\}$，其中 $\boldsymbol{V}_m\in\{\boldsymbol{U}_1,\boldsymbol{U}_2,\cdots,\boldsymbol{U}_L\}(m=1,2,\cdots,M)$。

如图 8 所示，在多元型样本编码中，位于第 m 个地址的数值超向量将使用排列运算 $M-m$ 次，在实际运算上体现为循环右移 $M-m$ 位。经过各自移位运算后的数值超向量再通过乘法进行绑定，得到最终的样本超维向量。

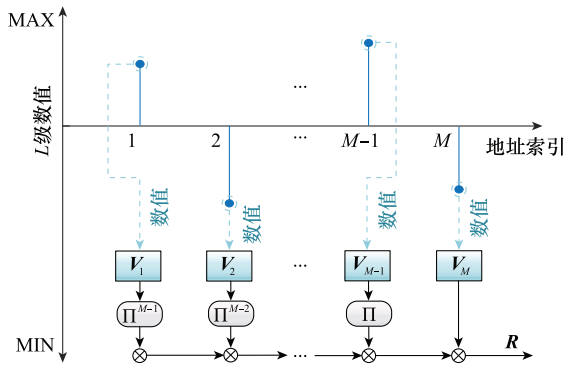


图 8 多元型样本编码

Fig. 8 Ngram-based encoding

该过程可以表示为如下形式：

$$\boldsymbol{R}=\Pi^{M-1}\boldsymbol{V}_1\oplus\Pi^{M-2}\boldsymbol{V}_2\oplus\cdots\oplus\boldsymbol{V}_M=\Pi(\cdots(\Pi(\Pi\boldsymbol{V}_1+\boldsymbol{V}_2)+\boldsymbol{V}_3)+\cdots)+\boldsymbol{V}_M \tag{14}$$

若令 $G(u)=\Pi^{u-1}\boldsymbol{V}_1\oplus\Pi^{u-2}\boldsymbol{V}_2\oplus\cdots\oplus\boldsymbol{V}_u$，式(14)的编码过程符合如下递推关系：

$$\begin{cases}G(1)=\boldsymbol{V}_1\\ G(u+1)=\Pi(G(u))\oplus\boldsymbol{V}_{u+1}\end{cases} \tag{15}$$

因为多元型样本编码只使用乘法和排列运算，所以编码的结果 $\boldsymbol{R}$ 保持与 $\boldsymbol{V}_m$ 相同的数据类型，不需要使用式(12)中的标准化运算。同时，由于乘法运算和排列运算都具有可逆性，编码结果 $\boldsymbol{R}$ 也是可逆的，通过乘法和排列运算的逆运算就可以恢复出每个数值超维向量。例如，恢复数值超维向量 $\boldsymbol{V}_1$ 可以通过如下方式实现：

$$\Pi^{1-M}[(\Pi^{M-1}V_1\oplus\Pi^{M-2}V_2\oplus\cdots\oplus V_M)\oplus(\Pi^{M-2}V_2\oplus\cdots\oplus V_M)]=\Pi^{1-M}[\Pi^{M-1}V_1\oplus(\Pi^{M-2}V_2\oplus\Pi^{M-2}V_2)\oplus\cdots\oplus(V_M\oplus V_M)]=\Pi^{1-M}(\Pi^{M-1}V_1)=V_1 \tag{16}$$

对比图 7 和图 8 可以发现，多元型样本编码由于省去了编码地址超维向量的过程，编码过程更加简洁，基于硬件友好性极强的排列运算，计算速度更快，占用计算资源更少。特别是，由于排列运算的性质，多元型样本编码可以高效、准确地保留顺序信息，在时间序列、频率序列等序列型数据的编码中应用广泛。

但是，相比于记录型样本编码在进行标准化处理前由加法得到的非二元型数据，多元型样本编码的结果仍然为二元型数据，样本超维向量每一位元素能够存储的信息量有限。特别是在重训练过程(第 3.4 节)中，更多的信息一般有助于微调模型，改善模型精度，进一步提高训练准确率和最终的测试准确率。值得说明的是，模型的最终效果取决于多方面因素，采用不同的样本编码方式对最终模型应用效果的影响不是绝对的。

3.3 训练与测试

基于上文对样本编码方式的介绍，本节介绍超维计算的基本训练与测试方法。

设有一个识别任务，训练样本共有 N_c 个种类，每类中各有 N_s 个样本，超维计算训练与测试过程的整体模型框架如图 9 所示。其中，具体的训练过程描述如下。

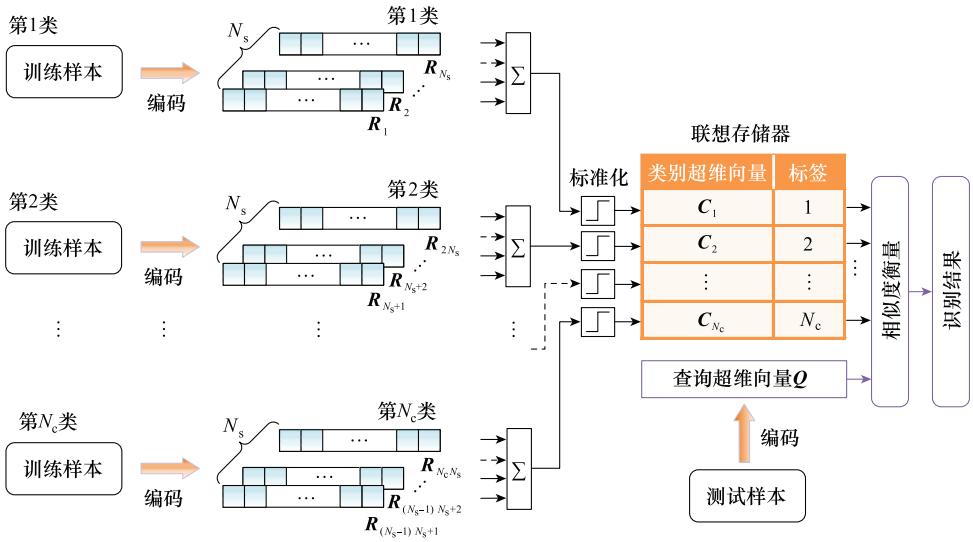


图 9 超维计算的训练与测试

Fig. 9 Training and test in hyperdimensional computing

步骤 1　使用第 3.1 节中的方法，通过随机选择将样本中的地址逐一编码为地址超维向量，通过连续映射将值域范围内的离散数值逐一编码为数值超维向量，编码好的地址与数值超维向量存储在被称为项目存储器的存储空间中，作为原料提供给后续算法取用。

步骤 2　使用第 3.2 节中的样本编码方式对每个训练样本进行编码，编码结果经过标准化处理后共得到 $N_c \cdot N_s$ 个超维向量，称为训练超维向量，记为 $\boldsymbol{R}_i(i=1,2,\cdots,N_c \cdot N_s)$。

步骤 3　将属于同一类的训练超维向量再次通过加法运算组成一个集合，记为 $\widetilde{\boldsymbol{C}}_k(k=1,2,\cdots,N_c)$，将 $\widetilde{\boldsymbol{C}}_k$ 经过标准化处理后得到 N_c 个超维向量，称为类别超向量，记为 $\boldsymbol{C}_k$ $(k=1,2,\cdots,N_c)$，$\boldsymbol{C}_k$ 与类别标签一起存储在被称为联想存储器的存储空间中，完成训练。

超维计算的测试过程同样也非常简洁，主要通过超维向量之间的相似度关系完成识别，具体步骤描述如下。

步骤 1　对测试样本使用项目存储器中的地址超维向量和数值超维向量，通过与训练样本相同的编码方式进行编码，编码结果称为查询超维向量，记为 $\boldsymbol{Q}$。

步骤 2　通过第 2.2 节中的相似度衡量方法，分别计算 $\boldsymbol{Q}$ 与联想存储器中的 N_c 个类别超维向量 $\boldsymbol{C}_k$ 之间的相似度。

步骤 3　最高相似度所对应的类别即为超维计算判别的测试样本所属类别。

上述模型框架使超维计算具有以下优点。

(1) 超快速学习过程：超维计算的训练过程在完成样本编码后，只需将属于同一类的训练超维向量相加即可，计算简单高效；

(2) 超快速测试过程：超维计算的测试过程基于超维向量之间的相似度做出判别，当采用二元数据类型进行编码时，使用汉明距离逐位判别向量元素是否相同即可完成相似度衡量，运算高效且运算量极低；

(3) 模型简单轻量：训练好的模型为 N_c 个类别超维向量，参数量少，并且超维向量元素一般为二元整数形式，模型占用存储空间小；

(4) 模型可验证性和可解释性优异：超维计算中所有的编码过程均可逆，模型透明；

(5) 小样本下的模型稳定性高：不需要通过迭代训练模型参数和不依赖于大数据驱动的特点，使超维计算在样本量较少时仍很好地能完成训练任务；

(6) 模型更新便捷快速：当新样本可用时，超维计算仅使用简单的编码和运算操作就可以完成对模型的补充和更新，进一步完善已有模型，不需要对模型参数进行重新训练。

3.4　重训练

本节介绍一种进一步完善和调整超维计算模型的方法，称为重训练方法。

当需要记忆的信息量较少时，大脑对不同种类信息之间的细微差异仍然能保持较好的区分能力。但是，随着记忆的信息不断增多以及受到其他错误信息的影响时，对某些特点不明显的信息，大脑的记忆也会逐渐模糊甚至错乱。同时，大脑对于新接收信息的印象也明显比早期信息更加深刻。类似的现象也出现在受大脑工作机制启发的超维计算模型中。

考虑到训练数据之间的差异，如果使用第 3.3 节中的训练方法，即每个训练数据只通过一次样本编码后直接作为类别超维向量的一部分，随着输入的训练样本数量增加，部分训练超维向量与其所属类别超维向量之间的相似性可能会逐渐降低。如果不进行进一步的处理，一些训练数据可能会丢失细节信息，甚至在相似度上更接近一个错误的类别，从而导致模型识别精度下降。这一问题在样本信噪比较低、类内差距较大以及部分训练样本标签类目错误等情况下将更加突出。因此，在训练完成后，对模型按照实际情况进行及时调整、补充和修正有时是必要的，重训练就是一种有效的方法。

重训练的概念由 Imani 等人在 2017 年提出[29]，通过增量更新类别超维向量，避免由于向单个类别超维向量添加大量样本超维向量而导致的信息容量问题和准确率下降问题，修正模型由于编码、样本质量、错误标签等各种原因导致的对某些训练样本的错误认知。

重训练的基本实现方法是在经过第 3.3 节训练过程后，再次计算每个训练超维向量 $\boldsymbol{R}_i$ 与每个类别超维向量 $\boldsymbol{C}_k$ 之间的相似度。这一操作相当于其他分类识别算法中计算训练准确率的过程，可以得到一个 $N_c \cdot N_c \cdot N_s$ 的相似度关系矩阵。此时，若某个属于 $\boldsymbol{C}_{\text{true}}$ 类别的训练样本 $\boldsymbol{R}_{\text{false}}$ 被错误分类为属于 $\boldsymbol{C}_{\text{false}}$ 类别，重训练将进行如下操作：

$$\begin{cases} \boldsymbol{C}_{\text{false}} = \boldsymbol{C}_{\text{false}} - \boldsymbol{R}_{\text{false}} \\ \boldsymbol{C}_{\text{true}} = \boldsymbol{C}_{\text{true}} + \boldsymbol{R}_{\text{false}} \end{cases} \tag{17}$$

式(17)的重训练过程在直观上很容易理解，相当于将被错误分类的训练样本从错误的类别中去除，并重新添加至正确的类别，加强模型对该样本的正确认知，去除错误认知。重训练过程可以迭代重复多次，直至模型的训练准确率达到收敛、稳定、最优或设定值。

相较于不使用重训练的超维计算模型，每次迭代中重训练额外增加了 $N_c \cdot N_c \cdot N_s$ 次相似度计算和 N_w 组加减法运算，其中 N_w 为每次重训练时被错误分类的训练样本个数，额外增加的运算量和所需要的计算资源不大。

重训练将训练样本重新作为模型输入的过程与训练连接主义模型时的迭代输入相似，但两者有着本质不同。对于连接主义模型中可能出现的过拟合问题，重训练从模型原理上直接避免了该问题。

超维计算中的迭代操作直接作用于类别超维向量，而不是像神经网络等连接主义模型中的迭代一样作用于复杂甚至不可解释的参数中，这使重训练不会导致训练模型的复杂性上升。得益于模型的透明性，重训练过程相当于从错误的类中自动删除错误分类的数据，并将其重新放入正确的类中。在超维空间中，这个调整的过程对各个类中已有的其他数据或信息影响很小。随着迭代次数的增加，越来越多的数据被模型正确学习和认知，当训练准确率稳定

后意味着只有一小部分训练数据仍在迭代，这些数据对整个训练模型的影响可以忽略不计，此时自然地停止重训练过程即可。在训练和测试过程中，相似度衡量将每个编码的样本数据与存储在类别超维向量中的整体全息信息进行比较，即使有少部分错误标注的训练数据，对最终准确率的影响也很小。

上述重训练方法可以高效迅速地提升训练准确率，但还主要存在如下两个问题：

(1) $\boldsymbol{R}_i$ 与 $\boldsymbol{C}_k$ 均为经过标准化处理后的二元数据类型，向量中每位元素的取值只有两种可能，每位保存信息量有限，若直接在二元数据上进行加减处理，可能反而会造成信息的缺失和混淆；

(2) 使用无权值的加减运算，训练准确率曲线收敛速度无法根据实际情况调节，可能会导致训练准确率收敛缓慢或震荡问题。

为解决上述问题，Imani 等人又进一步提出了一种基于超维计算的自适应学习方法(adaptive learning approach based on hyperdimensional computing，AdaptHD)[43]。对于第一个问题，AdaptHD 不再对使用记录型样本编码得到的样本超维向量和训练得到的类别超维向量进行标准化处理，保留加法运算产生的原始整数类型超维向量，即 $\tilde{\boldsymbol{R}}_i$ 和 $\tilde{\boldsymbol{C}}_k$，同时引入自适应变化的权值 α 与 $\tilde{\boldsymbol{R}}_i$ 逐位相乘，将式(17)转化为

$$\begin{cases}\tilde{\boldsymbol{C}}_{\text{false}}=\tilde{\boldsymbol{C}}_{\text{false}}-\alpha\tilde{\boldsymbol{R}}_{\text{false}}\\ \tilde{\boldsymbol{C}}_{\text{true}}=\tilde{\boldsymbol{C}}_{\text{true}}+\alpha\tilde{\boldsymbol{R}}_{\text{false}}\end{cases}\tag{18}$$

式中：$\tilde{\boldsymbol{R}}_i$ 和 $\tilde{\boldsymbol{C}}_k$ 均为非二元编码的超维向量，每位元素上包含的信息更加精确，在此基础上的加减运算也相对更加合理。α 类似于机器学习算法中学习率的作用，通过 α 可以控制每次对类别超维向量进行调节的量，进一步加快训练准确率收敛速度并提升最终模型的识别准确率。AdaptHD 中 α 的取值方法主要有 3 种。

方法 1：基于迭代的 AdaptHD，该方法通过每次迭代后的平均训练错误率 P_e 改变 α 的值，当平均错误率越大时，α 的值越大，反之则越小，其基本流程如图 10 所示。

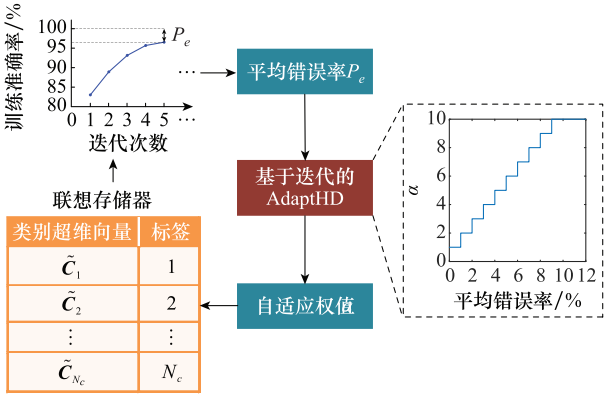


图 10 基于迭代的 AdaptHD

Fig. 10 Iteration-dependent AdaptHD

方法 2：基于数据的 AdaptHD，该方法通过样本超维向量与正确和错误类别超维向量之间相似度关系的差值 $\Delta\delta$，自适应地改变 α 的值。其中 $\Delta\delta$ 计算方法如下：

$$\Delta\delta=\delta_{\text{true}}-\delta_{\text{false}}=\cos(\tilde{\boldsymbol{R}}_{\text{false}},\tilde{\boldsymbol{C}}_{\text{correct}})-\cos(\tilde{\boldsymbol{R}}_{\text{false}},\tilde{\boldsymbol{C}}_{\text{false}})\tag{19}$$

为了使样本更接近正确的类别，远离错误的类别，当 $\Delta\delta$ 越小时，α 的值越大，反之越小，其基本流程如图 11 所示。

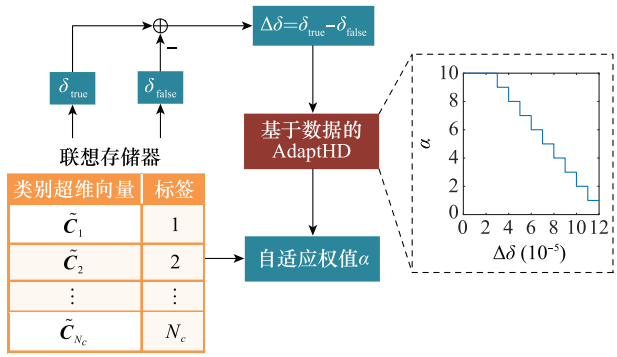


图 11 基于数据的 AdaptHD

Fig. 11 Data-dependent AdaptHD

方法 3：混合方法的 AdaptHD，结合方法 1 和方法 2，综合选取每次迭代中 α 的值。

由图 10 和图 11 可以看出，对于方法 1 和方法 2 中 α 的取值，文献[43]均采用阶梯式方法，即将浮点数形式的 P_e 和 $\Delta\delta$ 划分为多个区间，每个区间中使用对应的整数 α 取值。该方法具有良好的自适应效果，在训练样本与训练得到的模型有较大差异时可以明显加速训练准确率的收敛，提升最终的模型精度与识别准确率。

但是，AdaptHD 的缺点在于，阶梯式区间的划分、每个区间内 α 的取值和初始 α 最大值的选取都无标准、不固定，均需要按照实际应用场景调参，增加了额外的工作量。且 α 的最小值为 1，在模型中相当于只能将 $\tilde{\boldsymbol{R}}_{\text{false}}$ 完全去除或添加至类别超维向量，无法实现微调，即实现部分保留某个样本信息的功能。同时，由于平均准确率 P_e 一般是一个浮点数，且 $\tilde{\boldsymbol{R}}_i$ 和 $\tilde{\boldsymbol{C}}_k$ 的向量元素的数据类型为非二元整数，衡量相似度只能使用余弦距离，因此 $\Delta\delta$ 也将是一个浮点数。若 α 按照与 P_e 和 $\Delta\delta$ 之间的线性关系采用浮点数形式代入式(18)的重训练迭代过程中，类别超维向量中的数据会随之改变为浮点数类型，这显然大幅度增加了超维计算的运算成本和训练计算量，破坏了硬件友好性，不利于发挥超维计算的优势。

蒙版重训练[26]方法进一步解决了上述问题，其基本原理受 AdaptHD 启发，整体流程如图 12 所示。

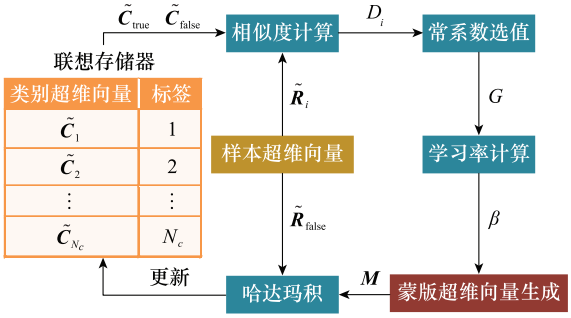


图 12　蒙版重训练流程

Fig. 12　Process of mask-retraining

具体流程描述如下。

首先，计算每个训练数据 $\tilde{\boldsymbol{R}}_i$ 与正确类别超向量 $\tilde{\boldsymbol{C}}_{\text{true}}$ 之间的相似度和与错误类别超向量 $\tilde{\boldsymbol{C}}_{\text{false}}$ 之间的相似度之差：

$$D_i=\delta_{\text{true}}-\delta_{\text{false}}=\cos(\tilde{\boldsymbol{R}}_i,\tilde{\boldsymbol{C}}_{\text{true}})-\cos(\tilde{\boldsymbol{R}}_i,\tilde{\boldsymbol{C}}_{\text{false}}),\quad i=1,2,\cdots,N_c\cdot N_s \tag{20}$$

被错误分类的样本超维向量 $\tilde{\boldsymbol{R}}_{\text{false}}$ 所对应的相似度之差 D_{false} 将是一个负数，即更靠近错误的类别，更远离正确的类别，重训练的目的则是使 D 的值越大越好，这与基于数据的 AdaptHD 中 $\Delta\delta$ 的概念是一致的。与 AdaptHD 中将 $\Delta\delta$ 按区间阶梯式选择权值 α 不同，蒙版重训练中的学习率以当前最大相似度关系差值 $\max(D_i)$ 为理想情况下的最优结果，通过 D_{false} 与该最优结果之间的差值调整学习率：

$$\beta=G\cdot(\max(D_i)-D_{\text{false}}) \tag{21}$$

式中：G 为常系数，类似放大倍数的作用，其值的确定可由最优的训练准确率收敛曲线决定。但此时，由于使用余弦距离运算，β 也是一个浮点数。为了解决浮点数不利于发挥超维计算优势的问题，蒙版重训练利用超维计算中信息全息分布式在超维向量各个元素中的概念，将浮点数转化为蒙版超维向量 $\boldsymbol{M}$ 的形式。当 $\beta\geqslant1$ 时，$\boldsymbol{M}=\langle\beta\rangle\cdot\text{ones}(1,d)$，其中$\langle\cdot\rangle$为四舍五入符号。此时，$\boldsymbol{M}$ 是一个元素数值全为$\langle\beta\rangle$的 d 维向量；当 $0\leqslant\beta<1$ 时，蒙版超维向量随机翻转 $\text{ones}(1,d)$ 中的 $d-\langle\beta d\rangle$ 位为 0 构成 $\boldsymbol{M}$。使用蒙版超维向量代替 β，将式(18)变换为

$$\begin{cases}\tilde{\boldsymbol{C}}_{\text{false}}=\tilde{\boldsymbol{C}}_{\text{false}}-M\circ\tilde{\boldsymbol{R}}_{\text{false}}\\ \tilde{\boldsymbol{C}}_{\text{true}}=\tilde{\boldsymbol{C}}_{\text{true}}+M\circ\tilde{\boldsymbol{R}}_{\text{false}}\end{cases} \tag{22}$$

式中：$\circ$为哈达玛积，实现两向量的元素按位相乘。

该重训练方法通过部分保留样本超维向量元素的方式实现对超维计算模型的细微调整，具有良好的自适应能力。同时，蒙版重训练没有过多的增加运算量，有效加速训练曲线收敛速度，提高识别精度的同时保持了现有超维计算重训练方法高效、快速的优势。

实际应用中，重训练在额外增加部分运算量的情况下可以有效提升训练模型精度，并提升最终的识别准确率，可以根据实际应用需求决定是否使用合适的重训练方法。

4　超维计算的典型应用

在基本阐述了超维计算的基本原理后，本章将介绍超维计算的 3 种典型应用场景，讨论超维计算在实际应用中的方法和效果，为超维计算在更多领域的应用提供思路和可行性。

4.1　语音信号识别

语音识别是人机交互、智能家居、机器翻译等领域中的重要环节之一，超维计算在轻量级物联网平台的语音识别中展现出超高能效和学习速度。文献[29]首次将超维计算用于语音识别中，提出了 VoiceHD 算法，处理流程如图 13 所示。

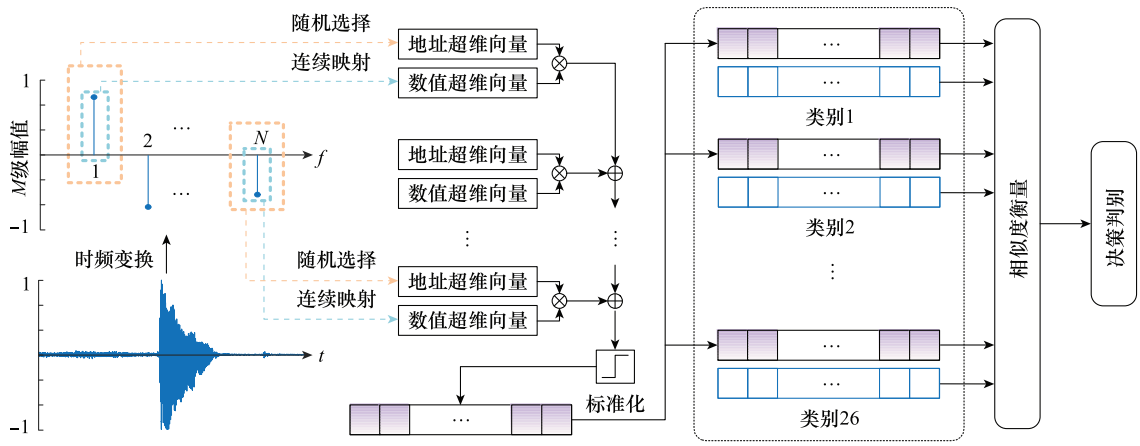


图 13　超维计算在语音信号识别中的处理流程

Fig. 13　Learning process of hyperdimensional computing in speech recognition

VoiceHD 基于 Isolet 数据集，对 26 个字母的语音信号进行训练与测试。该方法首先对语音时域信号通过其梅尔频率倒谱系数转化至频域中，设得到的频域谱图中共包含 N 个频率索引，各个频率上的幅值为范围在[−1,1]的

M 个离散值。文献中首先将 N 个频率索引视为 N 个独立地址，通过第 3.1 节中随机选择超维向量的方式得到 N 个相互独立且唯一的地址超维向量，记为 $\{\boldsymbol{ID}_1,\boldsymbol{ID}_2,\cdots,\boldsymbol{ID}_N\}$，通过连续映射产生一组数值超维向量与 M 个离散幅值一一对应，记为 $\{\boldsymbol{L}_1,\boldsymbol{L}_2,\cdots,\boldsymbol{L}_M\}$。使用第 3.2 节中图 7 的记录型编码方式，完成对样本的编码后进行训练。

在实验中，设置频率索引的个数 $N=617$。当 $M=10$ 时取得最大准确率 88.4%。增加式(17)的重训练对模型进行更新后，识别准确率进一步提升至 93.8%。在对比实验中，k 最邻近法(k-nearest neighbor，KNN)的识别准确率为 91.4%，低于超维计算；含有 48 个隐藏层的多任务深度神经网络识别准确率为 95.9%，略高于超维计算。但是，超维计算所需的计算资源极低，与准确率相近(93.6%)的具有 3 203 个神经元的深度神经网络相比，超维计算的能效高 11.9 倍，训练速度快 4.6 倍，测试速度快 5.3 倍。此外，文献中还将超维计算与一个简单的神经网络结合，将神经网络添加至超维计算的输出端进行判别，改进分类效果，进一步将识别准确率提升至 95.3%，且仍然比单纯使用神经网络的能效高 8.5 倍，训练速度快 2.9 倍，测试速度快 4.0倍。

VoiceHD 有效解决了物联网设备在有限计算资源和低电池容量的限制下，如何脱离云计算，实现设备本地的片上高效语音识别的问题。

4.2 语言识别

欧洲国家的语言一般都由字母组成，识别出任意一段文字具体是由哪个国家的语言写成的是自然语言处理中的一个经典问题。基于三字符组合频率直方图的方法是传统识别途径之一，对于 26 个字母和空格，共 27 个字符来说，每三个字符为一组共有 27^3 种可能的组合方式。该方法扫描一段文字中每三个字符的组合，将一段文字中各种组合情况的出现次数记录在 27^3 维的个数向量对应位置中，最终构成一个组合频率直方图，通过测试样本和训练样本的组合频率直方图相似度对比完成识别。但这种方法随着选择不同数量的字符组合作为识别依据时，直方图的维度将指数级上升，占用更多的存储空间和计算资源。超维计算可以有效解决上述问题[22,30-31]，基本处理流程如图 14 所示。

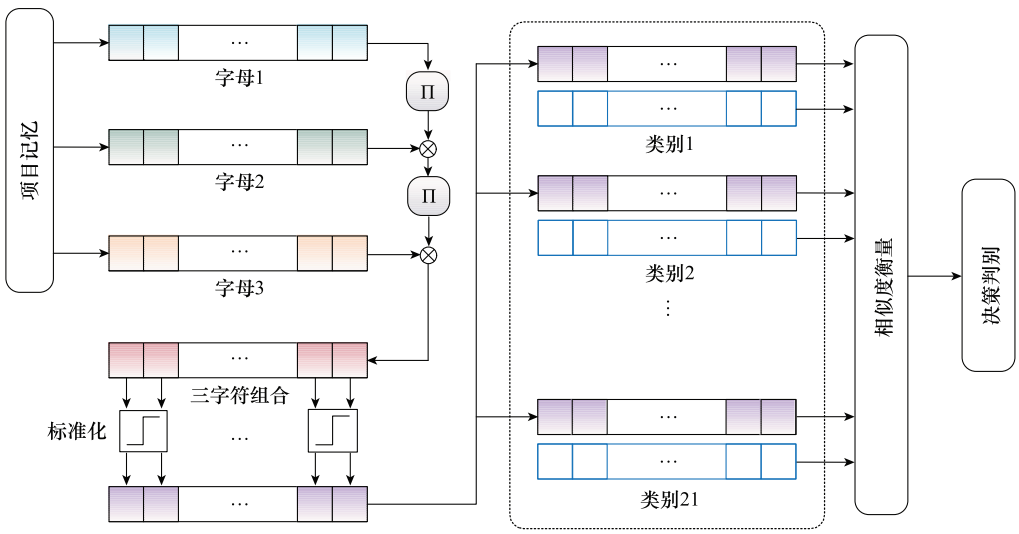


图 14 超维计算在语言识别中的处理流程

Fig. 14 Processing flow of hyperdimensional computing in language recognition

与计数构成直方图的方法不同，超维计算首先将 27 个字符作为独立符号，通过第 3.1 节中地址编码的方式随机从超维空间中选择 27 个超维向量与各个字符一一对应。然后每三个字符为一组，使用图 8 所示的多元型样本编码方式构成一个三字符组合超维向量。设字母 a、b、c 分别被编码为三个 d 维二进制超维向量，记为 $\boldsymbol{A}$、$\boldsymbol{B}$、$\boldsymbol{C}$，则三字符组合可以通过式(14)编码为 $\Pi^2\boldsymbol{A}\oplus\Pi\boldsymbol{B}\oplus\boldsymbol{C}=\Pi(\Pi\boldsymbol{A}\oplus\boldsymbol{B})\oplus\boldsymbol{C}$，编码结果仍然为一个 d 维超维向量。将一段文字中的所有三字符组合进行编码后通过加法运算构成一个和超维向量作为一个样本超维向量后，再将属于同一国家的文字全部相加构成类别超维向量，最终通过测试样本超维向量与类别超维向量之间的相似度对比完成识别。由于编码均在同一维度的超维空间中进行，所得的类别超维向量占用的存储空间和计算资源相对固定，使超维计算方法对于更多字符数量的组合有更好的泛化能力。

文献[22]对欧洲 21 国语言进行了分类，基于 Wortschatz 语料库[67]中约一百万字节的文本训练得到每个国家语言的样本超维向量，使用 Europarl 语料库[68]中每个国家各 1 000 个单独句子作为测试样本，最终的平均识别准确率为 96.7%，以传统机器学习一半的运算成本和能耗实现了仅比传统机器学习方法低 1.2%的准确率，且具有比传统机器学习方法更好的容错率。文献[31]将该算法移植至

新兴纳米技术集成电路芯片，实现了 20 000 个单独句子 98%的准确率。

4.3　图像识别

图像识别是人工智能算法的重要、热点领域之一，在目标识别、医学诊断、遥感勘测、自动驾驶等诸多应用场景中有着巨大研究价值。超维计算以其不同于现有人工智能算法的模型框架、特点和性质，在图像识别领域成为一种极具发展前景的新技术。

目前，超维计算直接识别图像的研究仍在起步阶段。文献[69]将超维计算应用于尺寸为 7×5 像素的小型黑白字母图像的识别，其编码方式如图 15 所示。

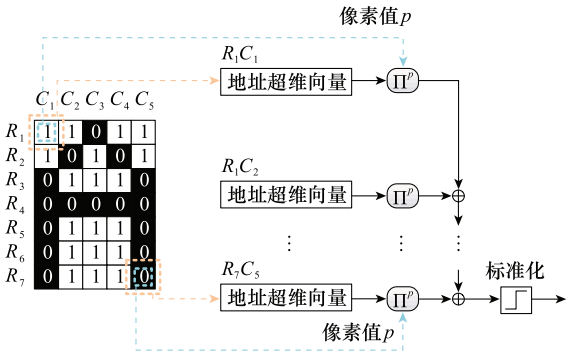


图 15　超维计算用于字母图像识别

Fig. 15　Character recognition using hyperdimensional computing

首先，生成 35 个相互独立且正交的地址超维向量代表每个像素的位置，根据各个位置的像素值是 0 还是 1 决定是否使用移位方式实现的排列运算。这种编码方式在文献[69]中被称为全息神经元（holographic graph neuron，HoloGN），该方法在二进制图像中相比生成数值超维向量再进行样本编码的方式更加高效。在此基础上，文献[69]和文献[70]进一步讨论了在字母图像失真等情况下超维计算的表现，通过二进制编码方式实现了优异识别效果并具有很强的鲁棒性，在 14.3%的失真率下仍然可以保持 90%左右的识别准确率，但识别的黑白字母图像尺寸较小且像素值仅有 0 和 1 两种取值。

更一般地，文献[32]将超维计算用于尺寸为 28×28 像素美国国家标准与技术研究院混合数据集（Mixed National Institute of Standards and Technolog Database，MNIST）手写数字图像数据集的识别中。文中采用双极编码方式，将灰度像素所在的共 784 个位置编码为地址超维向量，[0，255]的灰度值使用连续映射编码为数值超维向量，处理流程相当于将图像按行或按列直接展开为向量的形式，使用第 3.1 节中对语音信号记录型样本编码的方式编码手写数字图像，整体处理流程如图 16 所示。该算法在极短的训练时间内识别准确率可以达到约 90%，并通过生成对抗性输入的方式重训练超维计算模型，大幅增强了模型的鲁棒性和可靠性。

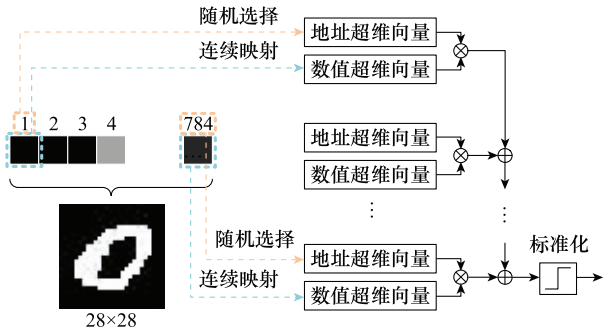


图 16　超维计算用于手写数字识别

Fig. 16　Hand-written digit recognition using hyperdimensional computing

从上述相关应用和研究中可以发现，超维计算更擅长处理一维结构（序列、矢量）的数据，而在处理二维结构（图像）的数据上优势暂时还不明显[28]。因此，现有图像识别应用中超维计算处理的对象大部分仍然是尺寸较小的简单图像，以验证可行性。对于更复杂图像的处理方式，有待进一步研究。

目前一种可行的方法是通过预处理得到图像中一维结构的特征信息，例如包络、投影、一维高分辨距离像（high range resolution profile，HRRP）、灰度直方图等，再输入超维计算进行识别。

文献[33]首先通过单演信号分析方法，将合成孔径雷达图像变换为基本不随目标配置、姿态角、俯仰角、随机噪声污染、遮挡等因素变化的多尺度高维解析信号，转化为一维向量的形式，然后使用主成分分析减少冗余数据后，使用张量积进行高维映射编码并输入超维计算。该方法在运动和静止目标获取与识别（moving and stationary target acquisition and recognition，MSTAR）公开数据集中，以极低的参数量和运算量实现了对合成孔径雷达图像 10 类目标的超快速、高准确率识别，与现有算法相比，在保证最终识别准确率的同时，进一步将运算量降低了 2～5 个数量级。

文献[26]中使用毫米波雷达对 6 种不同的实测人体姿态进行探测，并基于超维计算模型完成高效快速识别。结合雷达频谱图像的实际特点，文献[26]选择提取包络的方式保留频谱图中的微多普勒特征和空间能量关系，在大幅减少待处理数据量的同时，获得了一维结构的包络数据输入超维计算进行后续编码与识别。在完成一次初步训练后，进一步通过式(22)的蒙版重训练方法调整和完善模型，整体处理流程如图 17 所示。实验结果显示，蒙版重训练仅额外花费约 3.4%的总训练时间，将平均识别准确率从不使用重训练方法时的 83.2%有效提升至 92.9%。在对比实验中，超维计算的识别准确率超过了大部分传统分类器，并且与识别准确率相近的神经网络模型相比，超维计算模型的平均训练速度快 4.1 倍以上。在样本数较少和数据点有限等情况下，超维计算仍然表现出较强的稳定性和鲁棒性，准确率与训练时间上的优势将更加明显。

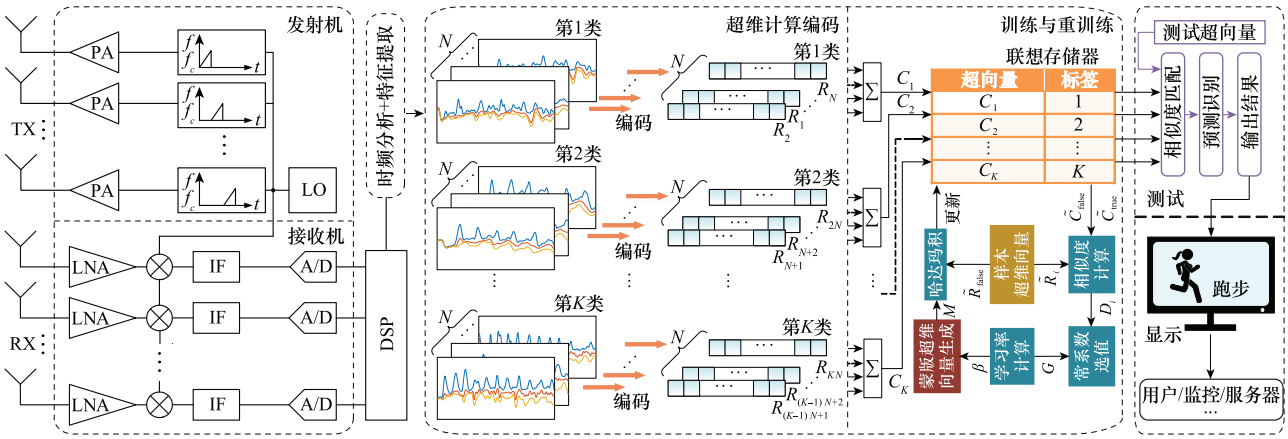


图 17 超维计算在雷达人体姿态识别中的处理流程

Fig. 17 Learning process of hyperdimensional computing in radar-based human activity recognition

5 超维计算存在的问题和发展方向

本章介绍超维计算在现阶段存在的问题和缺陷，并探讨超维计算在未来可能的问题解决方法与发展方向。

5.1 存在的问题

从前几章的叙述和介绍中可以发现，超维计算以其独特的轻量化模型结构，使用较少的计算资源，可以快速、高效、高鲁棒性地完成分类识别任务。但是，超维计算不是万能的，更不是毫无缺点的。超维计算在现阶段存在的问题主要体现在以下两点。

（1）最高识别准确率一般低于基于神经网络的连接主义模型。虽然在计算资源有限的平台中，在相同的训练时间，或在可用样本数量较少等情况下，超维计算的准确率可以接近甚至高于神经网络及其他分类识别算法。但是，在不考虑计算成本的情况下，除了例如文献[36]研究的癫痫病发作检测和文献[71]研究的乳腺癌识别等部分应用领域外，超维计算的识别准确率一般低于连接主义算法，且由于超维计算的模型和参数较为简单，在相对固定的模型框架下准确率也相对固定，最高准确率难以进一步提升。因此，文献[27]将超维计算的识别准确率称为是一种可接受的准确率。

（2）直接从复杂对象中自主提取特征的能力有限。由于在超维计算中使用高维分布式表示作为基本数据类型，实体信息被通过预先设定或选择好的编码规则一次映射到超维向量的每一个元素中，这种映射关系处理的是实体本身的直观数值特征，例如能量幅值、电流大小、灰度值等，抽象高层语义特征提取的能力有限。因此，超维计算目前较好的应用是识别语音信号、生物电信号等一维序列形式的数据，以及文字、DNA 序列等容易直接转化为数值向量形式的数据。对于图像等形式的数据，一般需要针对具体的分类识别任务进行特征提取操作并将数据转换为向量形式后，将超维计算作为特征的分类器使用。

5.2 未来发展方向

基于当前超维计算的概念、应用与研究进展，超维计算未来的发展方法可能主要有以下几点。

（1）更多的应用领域。作为一种新兴类人脑认知模型，超维计算的最直接发展方向是在更多的应用领域得到部署和应用，以其高硬件友好性、高鲁棒性、低样本量需求、快速学习过程和模型可解释性等优势特点，为各研究领域的不同任务提供一种可能的，甚至是不可替代的新选择。超维计算所能完成的任务不局限于分类识别，在环境感知、故障诊断、语义凝练、类比推理和逻辑关系表征等诸多方面，也有着大量模型层面的先天优势[72]。

（2）更优异的模型框架。超维计算的基本模型结构简洁，涉及参数量很少且显得相对固定单一。在未来的超维计算模型中，更高精度的地址、数值、样本编码方式，更精确的相似度衡量方法，更好的训练和测试流程，性能更好的重训练方法等，都可能进一步改进超维计算的现有模型识别效果，大幅提升超维计算模型的学习能力。

（3）更好的预处理方法。对于第 5.1 节中的第二个问题，在输入超维计算前提前使用一些高效的预处理方法是有效的。预处理可以实现减少数据量，去除冗余信息，保留核心特征数据等功能，这也对提高最终的识别准确率有一定的帮助。对于图像等数据而言，可以使用类似包络提取[26,35]、投影变换、稀疏表示[73]、主成分分析等特征提取方法将原始数据转化为向量形式或降低待处理数据量后再进行编码。甚至可以如文献[74]中通过哈希神经网络产生图像的高维表示，直接作为样本超维向量输入超维计算进行训练和测试，也可以取得良好的识别效果。但值得注意和思考的是，如何在通过额外计算量以完成预处理和保持超维计算的轻量化快速认知模型之间保持平衡。

（4）更通用的编码方式。超维计算的模型轻量且简洁，但从第 4 节可以看出，对于不同的学习任务，超维计算目前在样本编码环节仍然需要根据不同的输入数据类型选

择合适的编码方式和处理流程，若对于不同的输入数据类型能有相对比较通用、普适、精确的编码方式，可以使超维计算的模型更加统一，使用更加便捷快速，框架更易在不同的应用场景快速部署。

(5) 与其他算法的配合使用。多种算法相互结合，取长补短，发挥各自优势，是目前的热点研究方向之一。上述第三点中将超维计算与预处理方法结合也是其中的一种算法配合方式。同时，超维计算与其他分类识别算法，特别是连接主义模型，在思路上有着相似之处，例如都利用信息的高维表示作为运算和处理的基本环节。这也意味着在编码、训练、决策等环节，超维计算均有着和其他算法直接或间接配合使用的可能[75-78]。

(6) 相关硬件与芯片设计。随着超维计算的发展，相关或专用硬件与芯片设计技术也存在着研究与发展的前景[79-80]，可能仅使用包含大字长算数逻辑单元(arithmetic and logical unit，ALU)的集成电路就可以很好地完成认知与识别任务[81]。并行运算设备和硬件加速框架的发展同样有利于超维计算的发展。

6　结　论

本文首先从超维计算的发展历史出发，介绍了超维计算从雏形到正式提出的灵感来源、思路和过程。在简要介绍了超维空间和超维向量的基本背景知识后，详细阐述了超维计算的模型框架，包括具体的编码方式、训练测试流程以及进一步调整和完善模型的重训练方法。在此模型框架的基础上，给出了超维计算的典型应用示例，描述了超维计算处理信号、字符和图像数据的具体方法，为处理更多形式的数据提供参考。最后，探讨了超维计算在现阶段仍然存在的主要问题以及可能的发展方向。

超维计算作为类人脑新兴认知模型，利用超维向量作为信息的高维、随机、全息分布式表示，通过超维空间中的简单运算产生具有丰富性质的计算行为，进而产生记忆和认知，完成学习任务。在近年来以其运算成本低、学习速度快、硬件友好性高、鲁棒性强、模型可解释性好、样本需求量小等性质，得到了研究人员的广泛关注，在大量研究领域中展现出巨大潜力和价值。特别是在物联网领域和计算资源严格受限的情况下，超维计算为研究人员提供了一种可能的，甚至是不可替代的新选择。但是，超维计算有着诸多优点的同时，现阶段在识别准确率和自主提取特征能力等方面上仍然存在提升空间，值得在未来继续研究、完善和解决。

参考文献

[1] ROSENBLATT F. The perceptron: a probabilistic model for information storage and organization in the brain[J]. Psychological Review, 1958, 65(6): 386.

[2] RUMELHART D E, MCCLELLAND J L. Parallel distributed processing: explorations in the microstructure of cognition[M]. Cambridge: MIT Press, 1986.

[3] ANDERSON J A. A simple neural network generating an interactive memory[J]. Mathematical Biosciences, 1972, 14(3/4): 197 - 220.

[4] KOHONEN T. Correlation matrix memories[J]. IEEE Trans. on Computers, 1972, 100(4): 353 - 359.

[5] FUKUSHIMA K. Neocognitron: a self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position[J]. Biological Cybernetics, 1980, 36: 193 - 202.

[6] OLSHAUSEN B A, FIELD D J. Emergence of simple-cell receptive field properties by learning a sparse code for natural images[J]. Nature, 1996, 381(6583): 607 - 609.

[7] BELL A J, SEJNOWSKI T J. An information-maximization approach to blind separation and blind deconvolution[J]. Neural Computation, 1995, 7(6): 1129 - 1159.

[8] KRIZHEVSKY A, SUTSKEVER I, HINTON G E. Imagenet classification with deep convolutional neural networks[C]// Proc. of the Conference on Advances in Neural Information Processing Systems, 2012, 25: 1097 - 1105.

[9] HE K M, ZHANG X Y, REN S Q, et al. Deep residual learning for image recognition[C]// Proc. of the IEEE Conference on Computer Vision and Pattern Recognition, 2016: 770 - 778.

[10] SUTSKEVER I, VINYALS O, LE Q V. Sequence to sequence learning with neural networks[C]// Proc. of the Conference on Advances in Neural Information Processing Systems, 2014: 3104 - 3112.

[11] HINTON G E, DENG L, YU D, et al. Deep neural networks for acoustic modeling in speech recognition: the shared views of four research groups[J]. IEEE Signal Processing Magazine, 2012, 29(6): 82 - 97.

[12] LI F F, FERGUS R, PERONA P. One-shot learning of object categories[J]. IEEE Trans. on Pattern Analysis and Machine Intelligence, 2006, 28(4): 594 - 611.

[13] LIU Z, LI J G, SHEN Z Q, et al. Learning efficient convolutional networks through network slimming[C]// Proc. of the IEEE International Conference on Computer Vision, 2017: 2736 - 2744.

[14] ZEILER M D, FERGUS R. Visualizing and understanding convolutional networks[C]// Proc. of the European Conference on Computer Vision, 2014: 818 - 833.

[15] ZHANG Y, TIŇO P, LEONARDIS A, et al. A survey on neural network interpretability[J]. IEEE Trans. on Emerging Topics in Computational Intelligence, 2021, 5(5): 726 - 742.

[16] THOMAS A, DASGUPTA S, ROSING T. Theoretical foundations of hyperdimensional computing[J]. Journal of Artificial Intelligence Research, 2021, 72: 215 - 249.

[17] KANERVA P. Sparse distributed memory[M]. Cambridge: MIT Press, 1988.

[18] KANERVA P. Hyperdimensional computing: an introduction to computing in distributed representation with high-dimensional random vectors[J]. Cognitive Computation, 2009, 1(2): 139 - 159.

[19] MASSE N Y, TURNER G C, JEFFERIS G S X E. Olfactory

information processing in Drosophila[J]. Current Biology, 2009, 19(16): R700-R713.

[20] CARON S J C, RUTA V, ABBOTT L F, et al. Random convergence of olfactory inputs in the Drosophila mushroom body[J]. Nature, 2013, 497(7447): 113-117.

[21] SQUIRE L R. Memory and the hippocampus: a synthesis from findings with rats, monkeys, and humans[J]. Psychological Review, 1992, 99(2): 195-231.

[22] RAHIMI A, KANERVA P, RABAEY J M. A robust and energy-efficient classifier using brain-inspired hyperdimensional computing[C]// Proc. of the International Symposium on Low Power Electronics and Design, 2016: 64-69.

[23] ZHANG S, WANG R, ZHANG J J, et al. Assessing robustness of hyperdimensional computing against errors in associative memory[C]// Proc. of the IEEE 32nd International Conference on Application-specific Systems, Architectures and Processors, 2021: 211-217.

[24] HERSCHE M, LIPPUNER S, KORB M, et al. Near-channel classifier: symbiotic communication and classification in high-dimensional space[J]. Brain Informatics, 2021, 8(1): 16.

[25] KARUNARATNE G, SCHMUCK M, LE GALLO M, et al. Robust high-dimensional memory-augmented neural networks[J]. Nature Communications, 2021, 12(1): 2468.

[26] YAO Y R, LIU W B, ZHANG G, et al. Radar-based human activity recognition using hyperdimensional computing[J]. IEEE Trans. on Microwave Theory and Techniques, 2022, 70(3): 1605-1619.

[27] GE L L, PARHI K K. Classification using hyperdimensional computing: a review[J]. IEEE Circuits and Systems Magazine, 2020, 20(2): 30-47.

[28] HASSAN E, HALAWANI Y, MOHAMMAD B, et al. Hyper-dimensional computing challenges and opportunities for AI applications[J]. IEEE Access, 2021, 10: 97651-97664.

[29] IMANI M, KONG D, RAHIMI A, et al. VoiceHD: hyperdimensional computing for efficient speech recognition[C]// Proc. of the IEEE International Conference on Rebooting Computing, 2017.

[30] JOSHI A, HALSETH J T, KANERVA P. Language geometry using random indexing[C]// Proc. of the International Symposium on Quantum Interaction, 2016: 265-274.

[31] IMANI M, HWANG J, ROSING T, et al. Low-power sparse hyperdimensional encoder for language recognition[J]. IEEE Design and Test, 2017, 34(6): 94-101.

[32] MA D N, GUO J M, JIANG Y, et al. HDTest: differential fuzz testing of brain-inspired hyperdimensional computing[C]// Proc. of the ACM/IEEE Design Automation Conference, 2021: 391-396.

[33] YAO Y R, LIU W B, ZHANG G, et al. Fast SAR image recognition via hyperdimensional computing using monogenic mapping[J]. IEEE Geoscience and Remote Sensing Letters, 2022, 19: 4508205.

[34] RAHIMI A, KANERVA P, BENINI L, et al. Efficient biosignal processing using hyperdimensional computing: network templates for combined learning and classification of ExG signals[J]. Proceedings of the IEEE, 2018, 107(1): 123-143.

[35] RAHIMI A, BENATTI S, KANERVA P, et al. Hyperdimensional biosignal processing: a case study for EMG-based hand gesture recognition[C]// Proc. of the IEEE International Conference on Rebooting Computing, 2016.

[36] BURRELLO A, SCHINDLER K, BENINI L, et al. Hyperdimensional computing with local binary patterns: one-shot learning of seizure onset and identification of ictogenic brain regions using short-time iEEG recordings[J]. IEEE Trans. on Biomedical Engineering, 2019, 67(2): 601-613.

[37] KIM Y, IMANI M, MOSHIRI N, et al. GenieHD: efficient DNA pattern matching accelerator using hyperdimensional computing[C]// Proc. of the Design, Automation and Test in Europe Conference and Exhibition, 2020: 115-120.

[38] CHANG C Y, CHUANG Y, CHANG E J, et al. MulTa-HDC: a multi-task learning framework for hyperdimensional computing[J]. IEEE Trans. on Computers, 2021, 70(8): 1269-1284.

[39] RÄSÄNEN O J, SAARINEN J P. Sequence prediction with sparse distributed hyperdimensional coding applied to the analysis of mobile phone use patterns[J]. IEEE Trans. on Neural Networks and Learning Systems, 2015, 27(9): 1878-1889.

[40] RASANEN O, KAKOUROS S. Modeling dependencies in multiple parallel data streams with hyperdimensional computing[J]. IEEE Signal Processing Letters, 2014, 21(7): 899-903.

[41] SCHLEGEL K, MIRUS F, NEUBERT P, et al. Multivariate time series analysis for driving style classification using neural networks and hyperdimensional computing[C]// Proc. of the IEEE Intelligent Vehicles Symposium, 2021: 602-609.

[42] RAHIMI A, KANERVA P, MILLAN J R, et al. Hyperdimensional computing for noninvasive brain-computer interfaces: blind and one-shot classification of EEG error-related potentials[C]// Proc. of the ACM/EAI International Conference on Bio-inspired Information and Communications Technologies, 2017: 19-26.

[43] IMANI M, MORRIS J, BOSCH S, et al. AdaptHD: adaptive efficient training for brain-inspired hyperdimensional computing[C]// Proc. of the IEEE Biomedical Circuits and Systems Conference, 2019.

[44] RAHIMI A, DATTA S, KLEYKO D, et al. High-dimensional computing as a nanoscalable paradigm[J]. IEEE Trans. on Circuits and Systems I: Regular Papers, 2017, 64(9): 2508-2521.

[45] IMANI M, MESSERLY J, WU F, et al. A binary learning framework for hyperdimensional computing[C]// Proc. of the Design, Automation and Test in Europe Conference & Exhibition, 2019: 126-131.

[46] SCHMUCK M, BENINI L, RAHIMI A. Hardware optimizations of dense binary hyperdimensional computing: rematerialization of hypervectors, binarized bundling, and combinational associative memory[J]. ACM Journal on Emerging Technologies in Computing Systems, 2019, 15(4): 32.

[47] SALAMAT S, IMANI M, ROSING T. Accelerating hyperdimensional computing on FPGAs by exploiting computational

reuse[J]. IEEE Trans. on Computers, 2020, 69(8): 1159 - 1171.

[48] LI H, WU T F, RAHIMI A, et al. Hyperdimensional computing with 3D VRRAM in-memory kernels: device-architecture co-design for energy-efficient, error-resilient language recognition[C]// Proc. of the IEEE International Electron Devices Meeting, 2016.

[49] WU T F, LI H, HUANG P C, et al. Brain-inspired computing exploiting carbon nanotube FETs and resistive RAM: hyperdimensional computing case study[C]// Proc. of the IEEE International Solid-State Circuits Conference, 2018: 492 - 494.

[50] KARUNARATNE G, LE-GALLO M, CHERUBINI G, et al. In-memory hyperdimensional computing[J]. Nature Electronics, 2020, 3(6): 327 - 337.

[51] HOPFIELD J J. Neural networks and physical systems with emergent collective computational abilities[J]. Proceedings of the National Academy of Sciences, 1982, 79(8): 2554 - 2558.

[52] ACKLEY D H, HINTON G E, SEJNOWSKI T J. A learning algorithm for Boltzmann machines[J]. Cognitive Science, 1985, 9(1): 147 - 169.

[53] HINTON G E, OSINDERO S, TEH Y W. A fast learning algorithm for deep belief nets[J]. Neural Computation, 2006, 18(7): 1527 - 1554.

[54] HINTON G E, ANDERSON J A. Parallel models of associative memory: updated edition[M]. New York: Psychology Press, 2014.

[55] MINSKY M, PAPERT S. Perceptrons[M]. Cambridge: MIT Press, 1969.

[56] WERBOS P J. Backpropagation through time: what it does and how to do it[J]. Proceedings of the IEEE, 1990, 78(10): 1550 - 1560.

[57] RUMELHART D E, HINTON G E, WILLIAMS R J. Learning representations by back-propagating errors[J]. Nature, 1986, 323(6088): 533 - 536.

[58] LECUN Y, BOSER B, DENKER J S, et al. Backpropagation applied to handwritten zip code recognition[J]. Neural Computation, 1989, 1(4): 541 - 551.

[59] HINTON G E. Mapping part-whole hierarchies into connectionist networks[J]. Artificial Intelligence, 1990, 46(1/2): 47 - 75.

[60] SMOLENSKY P. Tensor product variable binding and the representation of symbolic structures in connectionist systems[J]. Artificial Intelligence, 1990, 46(1/2): 159 - 216.

[61] PLATE T A. Holographic reduced representations[J]. IEEE Trans. on Neural Networks, 1995, 6(3): 623 - 641.

[62] KANERVA P. Binary spatter-coding of ordered *k*-tuples[C]// Proc. of the International Conference on Artificial Neural Networks, 1996: 869 - 873.

[63] GAYLER R W. Multiplicative binding representation operators and analogy[C]// Proc. of the Workshop on Advances in Analogy Research: Integration of Theory and Data from the Cognitive Computational and Neural Sciences, 1998.

[64] PLATE T A. Holographic reduced representations: distributed representation for cognitive structures[M]. Stanford: CSLI, 2003.

[65] LEVY S D, GAYLER R. Vector symbolic architectures: a new building material for artificial general intelligence[C]// Proc. of the Conference on Artificial General Intelligence, 2008: 414 - 418.

[66] IMANI M, HUANG C, KONG D, et al. Hierarchical hyperdimensional computing for energy efficient classification[C]// Proc. of the ACM/ESDA/IEEE Design Automation Conference, 2018.

[67] QUASTHOFF U, RICHTER M, BIEMANN C. Corpus portal for search in monolingual corpora[C]// Proc. of the International Conference on Language Resources and Evaluation, 2006: 1799 - 1802.

[68] KOEHN P. Europarl: a parallel corpus for statistical machine translation[C]// Proc. of the Machine Translation Summit X, 2005: 79 - 86.

[69] KLEYKO D, OSIPOV E, SENIOR A, et al. Holographic graph neuron: a bioinspired architecture for pattern processing[J]. IEEE Trans. on Neural Networks and Learning Systems, 2016, 28(6): 1250 - 1262.

[70] MANABAT A X, MARCELO C R, QUINQUITO A L, et al. Performance analysis of hyperdimensional computing for character recognition[C]// Proc. of the International Symposium on Multimedia and Communication Technology, 2019.

[71] IMANI M, KIM Y, WORLEY T, et al. HDCluster: an accurate clustering using brain-inspired high-dimensional computing[C]// Proc. of the Design, Automation and Test in Europe Conference & Exhibition, 2019: 1591 - 1594.

[72] KLEYKO D, RAHIMI A, RACHKOVSKIJ D A, et al. Classification and recall with binary hyperdimensional computing: tradeoffs in choice of density and mapping characteristics[J]. IEEE Trans. on Neural Networks and Learning Systems, 2018, 29(12): 5880 - 5898.

[73] IMANI M, SALAMAT S, KHALEGHI B, et al. SparseHD: algorithm-hardware co-optimization for efficient high-dimensional computing[C]// Proc. of the IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines, 2019: 190 - 198.

[74] MITROKHIN A, SUTOR P, SUMMERS-STAY D, et al. Symbolic representation and learning with hyperdimensional computing[J]. Frontiers in Robotics and AI, 2020, 7: 63.

[75] ALONSO P, SHRIDHAR K, KLEYKO D, et al. HyperEmbed: tradeoffs between resources and performance in NLP tasks with hyperdimensional computing enabled embedding of n-gram statistics[C]// Proc. of the International Joint Conference on Neural Networks, 2021.

[76] KLEYKO D, KHEFFACHE M, FRADY E P, et al. Density encoding enables resource-efficient randomly connected neural networks[J]. IEEE Trans. on Neural Networks and Learning Systems, 2020, 32(8): 3777 - 3783.

[77] MORRIS J, LUI H W, STEWART K, et al. HyperSpike: hyperdimensional computing for more efficient and robust spiking neural networks[C]// Proc. of the 2022 Design, Automation & Test in Europe Conference & Exhibition, 2022: 664 - 669.

[78] KLEYKO D, FRADY E P, KHEFFACHE M, et al. Integer echo state networks: efficient reservoir computing for digital hardware[J]. IEEE Trans. on Neural Networks and Learning Systems, 2022, 33(4): 1688 - 1701.

[79] HSU P K, YU S. In-memory 3D NAND flash hyperdimensional computing engine for energy-efficient SARS-CoV-2 genome sequencing[C] // Proc. of the IEEE International Memory Workshop, 2022.

[80] DUBREUIL T, AMARI P, BARRAUD S, et al. A novel 3D 1T1R RRAM architecture for memory-centric hyperdimensional computing[C]// Proc. of the IEEE International Memory Workshop, 2022.

[81] DATTA S, ANTONIO R A G, ISON A R S, et al. A programmable hyper-dimensional processor architecture for human-centric IoT[J]. IEEE Journal on Emerging and Selected Topics in Circuits and Systems, 2019, 9(3): 439 - 452.

作者简介

刘文波(1968—),女,教授,博士,主要研究方向为控制与测量技术、信号处理、深度学习、非线性系统控制。

姚翼荣(1998—),男,硕士研究生,主要研究方向为信号处理、目标检测与识别、机器学习。

张 弓(1964—),男,教授,博士,主要研究方向为雷达信号处理、目标识别、SAR 图像处理。

胡 文(1979—),男,副教授,博士,主要研究方向为雷达信号处理、新体制雷达。